

Deep Reinforcement Learning and Control

MBRL with Multimodal Dynamics

Spring 2025, CMU 10-403

Instructors: Katerina Fragkiadaki & Aviral Kumar



Used materials

Today's slides are based on Stefano Ermon's course on deep generative models and on the author's slides of the Diffuser paper ICML 2022, the blogpost of Calvin Luo on Understanding Diffusion Models: A Unified Perspective, and the excellent course of diffusion models and applications CS492 of Minhyuk Sung

Planning with Diffusion for Flexible Behavior Synthesis

Michael Janner^{*}, Yilun Du^{*}, Joshua Tenenbaum, and Sergey Levine

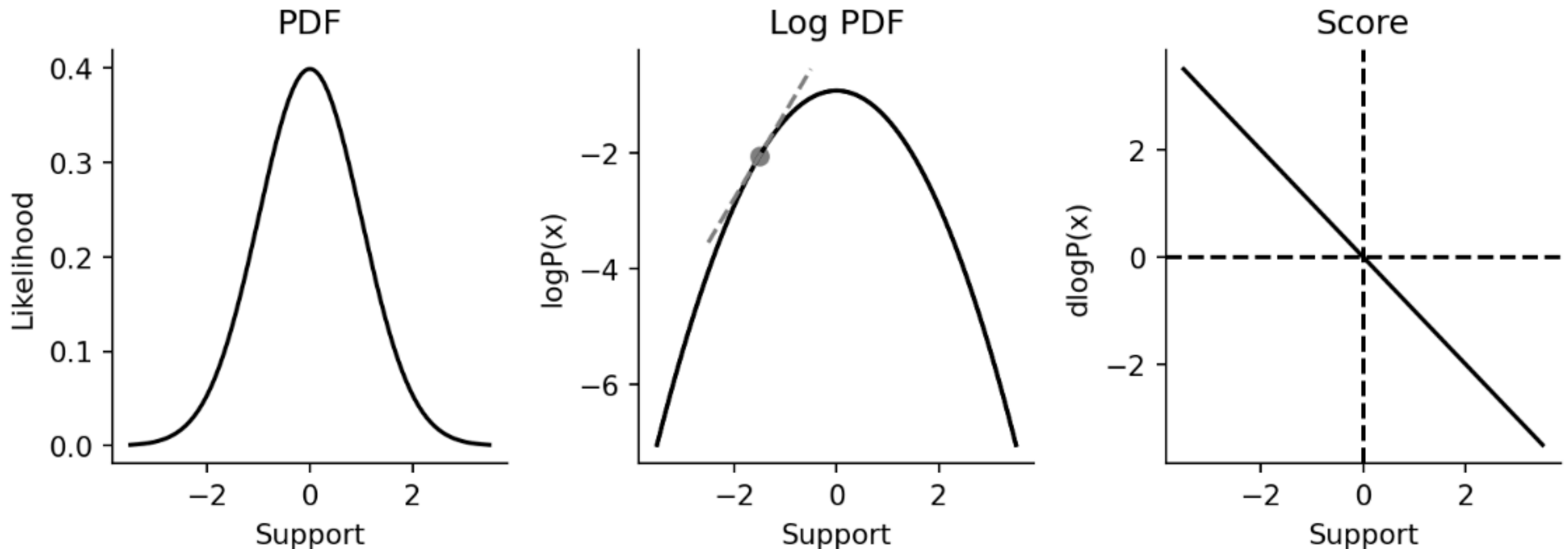
Model-based RL with diffusion models:

1. Train a state-action **trajectory** diffusion model.
2. Guide it with learnt value functions towards high value trajectory samples

Understanding Guidance in Diffusion Models

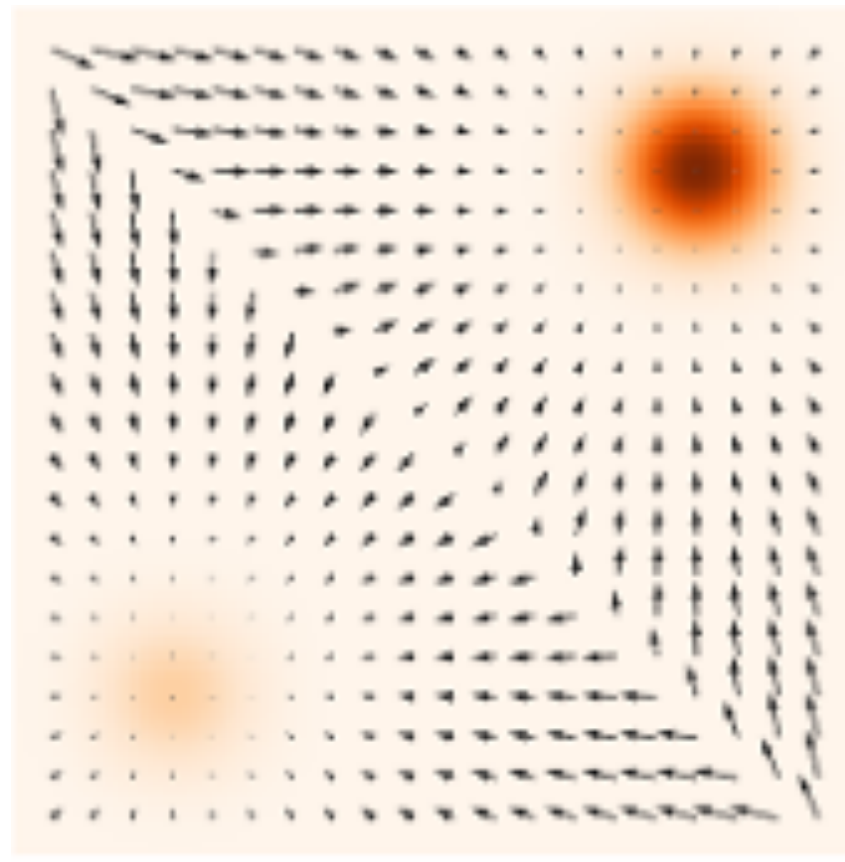
Score function

Score: The gradient of the log-probability density w.r.t. the input dimensions $\nabla_{\mathbf{x}} \log p(\mathbf{x})$



Score function

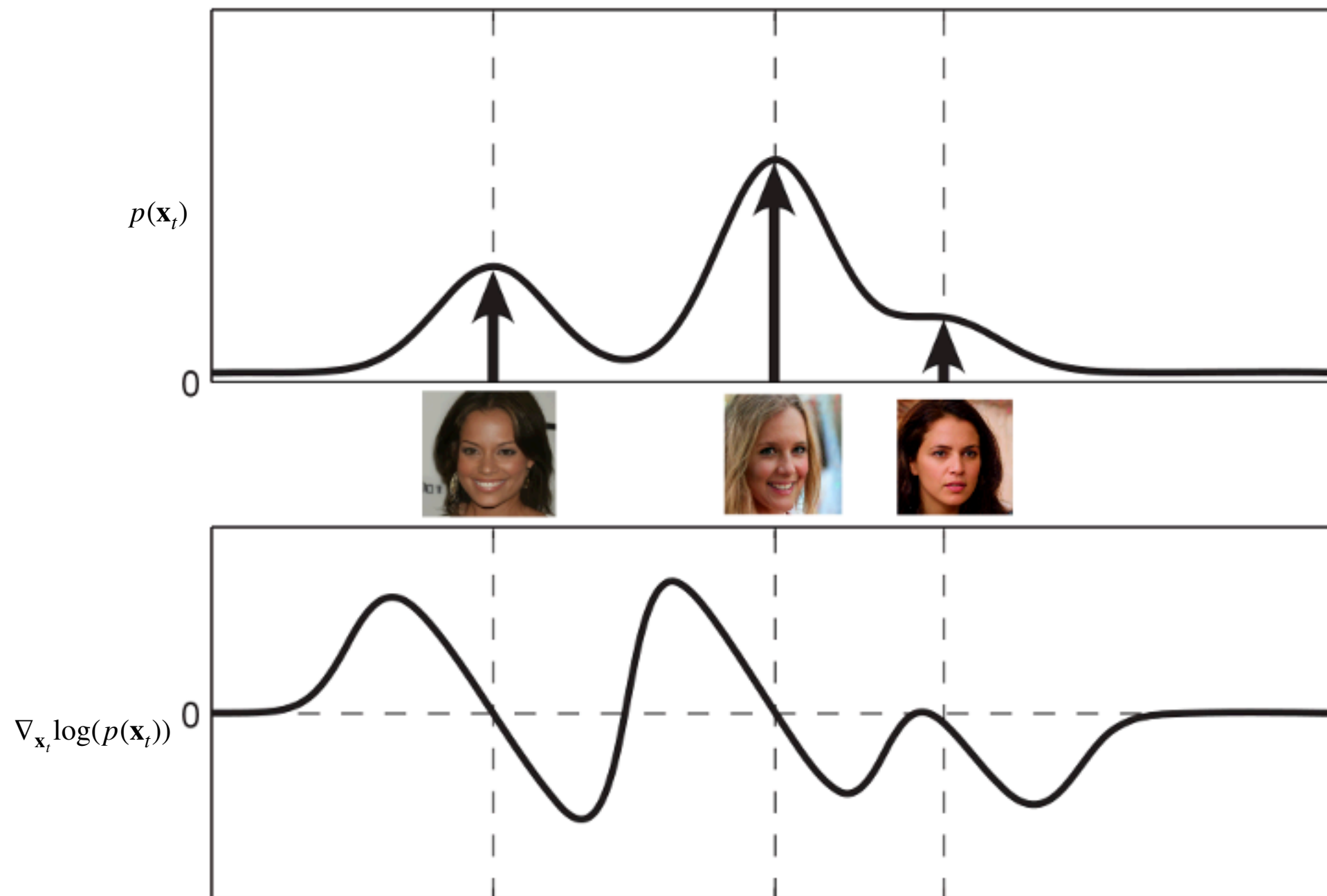
Score: The gradient of the log-probability density w.r.t. the input dimensions $\nabla_{\mathbf{x}} \log p(\mathbf{x})$



(pdf and score)

The score function defines a vector field over the entire data space, pointing towards the modes.

Score function



Adapted from Raphan and Simoncelli, 2010

Langevin Dynamics

Why is the **score** function important?

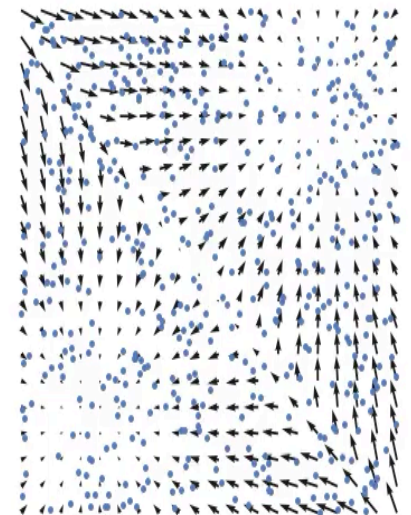
Even without knowing $q(\boldsymbol{x})$, if we have the score function $\nabla_{\boldsymbol{x}} \log q(\boldsymbol{x})$, we can sample from the distribution $q(\boldsymbol{x})$ using **Langevin dynamics**!

Langevin Dynamics

- Sample $\mathbf{x}$ from a prior distribution.
- Iterate the following procedure T steps:

$$\mathbf{x} \leftarrow \mathbf{x} + \eta \nabla_{\mathbf{x}} \log q(\mathbf{x}) + \sqrt{2\eta} \boldsymbol{\varepsilon}, \quad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

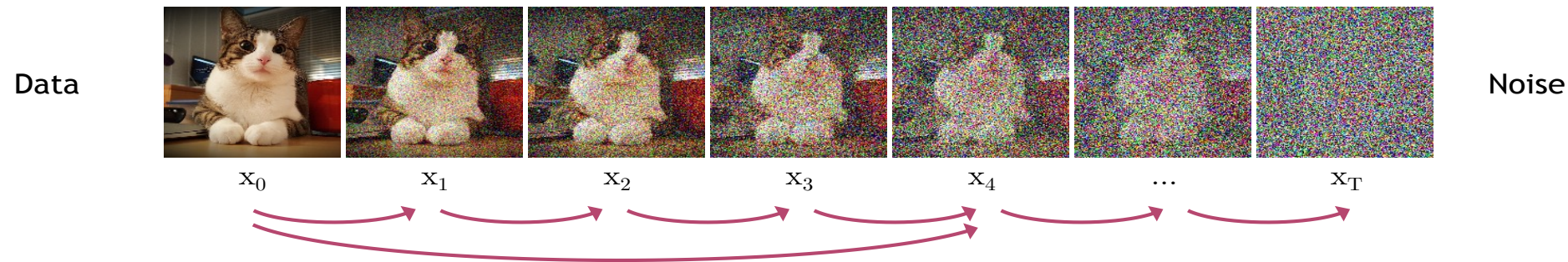
- It converges to $q(\mathbf{x})$ when $\eta \rightarrow 0$ and $T \rightarrow \infty$
- Note that it only uses the **score** information $\nabla_{\mathbf{x}} \log q(\mathbf{x})$, not $q(\mathbf{x})$.



Follow the scores

$$\mathbf{x} \leftarrow \mathbf{x} + \eta \nabla_{\mathbf{x}} \log q(\mathbf{x})$$

Reminder: $q(\mathbf{x}_t | \mathbf{x}_0)$ is Gaussian



For an arbitrary sample $\mathbf{x}_t \sim q(\mathbf{x}_t | \mathbf{x}_0)$:

$$\begin{aligned}
 \mathbf{x}_t &= \sqrt{\alpha_t} \mathbf{x}_{t-1} + \sqrt{1 - \alpha_t} \boldsymbol{\epsilon}_{t-1}^* \\
 &= \sqrt{\alpha_t} \left(\sqrt{\alpha_{t-1}} \mathbf{x}_{t-2} + \sqrt{1 - \alpha_{t-1}} \boldsymbol{\epsilon}_{t-2}^* \right) + \sqrt{1 - \alpha_t} \boldsymbol{\epsilon}_{t-1}^* \\
 &= \sqrt{\alpha_t \alpha_{t-1}} \mathbf{x}_{t-2} + \sqrt{\alpha_t - \alpha_t \alpha_{t-1}} \boldsymbol{\epsilon}_{t-2}^* + \sqrt{1 - \alpha_t} \boldsymbol{\epsilon}_{t-1}^* \\
 &= \sqrt{\alpha_t \alpha_{t-1}} \mathbf{x}_{t-2} + \sqrt{\sqrt{\alpha_t - \alpha_t \alpha_{t-1}}^2 + \sqrt{1 - \alpha_t}^2} \boldsymbol{\epsilon}_{t-2}^* \\
 &= \sqrt{\alpha_t \alpha_{t-1}} \mathbf{x}_{t-2} + \sqrt{\alpha_t - \alpha_t \alpha_{t-1} + 1 - \alpha_t} \boldsymbol{\epsilon}_{t-2}^* \\
 &= \sqrt{\alpha_t \alpha_{t-1}} \mathbf{x}_{t-2} + \sqrt{1 - \alpha_t \alpha_{t-1}} \boldsymbol{\epsilon}_{t-2}^* \\
 &= \dots \\
 &= \sqrt{\prod_{i=1}^t \alpha_i} \mathbf{x}_0 + \sqrt{1 - \prod_{i=1}^t \alpha_i} \boldsymbol{\epsilon}_0 \\
 &= \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}_0
 \end{aligned}$$

The sum of two independent Gaussian random variables is a Gaussian with mean being the sum of the two means, and variance being the sum of the two variances

$$\sim \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I})$$

- What is the **score** of $q(\mathbf{x}_t | \mathbf{x}_0)$: $\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t | \mathbf{x}_0)$?

Score function

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I})$$

The **score** of $q(\mathbf{x}_t|\mathbf{x}_0)$:

$$\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t|\mathbf{x}_0) = \nabla_{\mathbf{x}} \left(-\frac{\|\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\mathbf{x}_0\|^2}{2(1 - \bar{\alpha}_t)} \right) = -\frac{\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\mathbf{x}_0}{1 - \bar{\alpha}_t}$$

Score function

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I})$$

$$\boldsymbol{\varepsilon}_t = \frac{1}{\sqrt{1 - \bar{\alpha}_t}}(\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\mathbf{x}_0)$$

$$\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t|\mathbf{x}_0) = -\frac{\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\mathbf{x}_0}{1 - \bar{\alpha}_t} = -\frac{\boldsymbol{\varepsilon}_t}{\sqrt{1 - \bar{\alpha}_t}}$$

Score function

$$\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t | \mathbf{x}_0) = -\frac{\mathbf{x}_t - \sqrt{\bar{\alpha}_t} \mathbf{x}_0}{1 - \bar{\alpha}_t} = -\frac{\boldsymbol{\varepsilon}_t}{\sqrt{1 - \bar{\alpha}_t}}$$

*The **noise predictor** $\hat{\boldsymbol{\varepsilon}}_\theta(\mathbf{x}_t, t)$ can be interpreted as predicting the score $\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t | \mathbf{x}_0)$, up to a scaling factor.*

- The score function measures how to move in data space to maximize the log probability.
- Since the source noise is added to a clean datapoint, moving in its opposite direction would be the best update to increase the log probability.

Controlling the generation process with classifier guidance

Assume that data x and class label y pairs are given:

$$p(x_t, y) = p(\mathbf{x}_t)p(y|\mathbf{x}_t)$$



Can I generate data samples from the distribution $p(x, y)$ without training a conditional generative model?

Controlling the generation process with classifier guidance

At each timestep t , we are interested in the **score** function $\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t, y)$:

$$\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t, y) = \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log p(y|\mathbf{x}_t)$$

$$= -\frac{1}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t) + \nabla_{\mathbf{x}_t} \log p(y|\mathbf{x}_t)$$

$$= -\frac{1}{\sqrt{1-\bar{\alpha}_t}} \left(\boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t) - \sqrt{1-\bar{\alpha}_t} \nabla_{\mathbf{x}_t} \log p(y|\mathbf{x}_t) \right)$$

New noise!

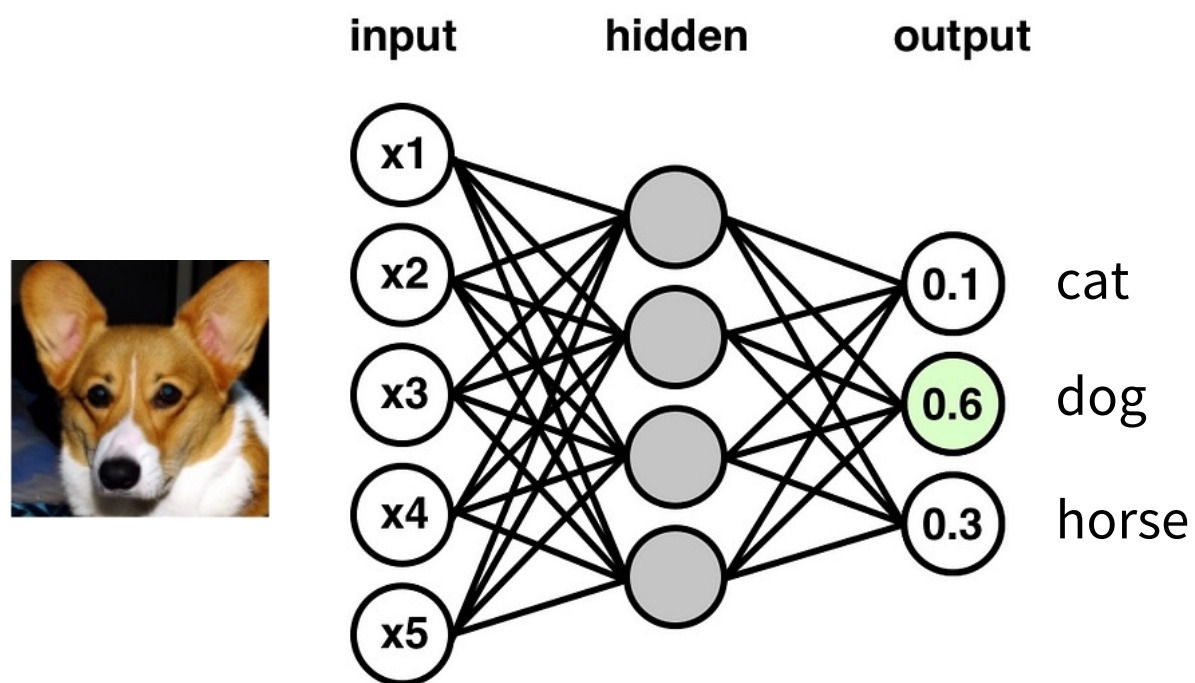
How to
compute this?

- Modelled by an unconditional diffusion model
- Computed by the gradient of the log-likelihood of a classifier

Controlling the generation process with classifier guidance

How to compute $\nabla_{\mathbf{x}_t} \log p(y|\mathbf{x}_t)$?

How to learn $p(y|\mathbf{x}_t)$? Train a **classification** network!



Any classifier $p(y | \mathbf{x}_t)$ used for guidance to provide a useful signal all the way through the sampling process needs to be trained on both clean and noised images.

Controlling the generation process with classifier guidance

How to compute $\nabla_{\mathbf{x}_t} \log p(y|\mathbf{x}_t)$?

- Train a **classifier** $p_\phi(y|\mathbf{x}_t)$, taking noisy data $\mathbf{x}_t$ (and time step t) as input and classifying it.
- Update the noise prediction $\hat{\boldsymbol{\epsilon}}_\theta(\mathbf{x}_t, t)$ as follows:

$$\tilde{\boldsymbol{\epsilon}}_\theta(\mathbf{x}_t, y, t) = \hat{\boldsymbol{\epsilon}}_\theta(\mathbf{x}_t, t) - \sqrt{1 - \bar{\alpha}_t} \nabla_{\mathbf{x}_t} \log p_\phi(y|\mathbf{x}_t)$$

Controlling the generation process with classifier guidance

The **strength** of the classifier guidance can be controlled by adding a **weight** $\omega \geq 1$:

$$\tilde{\mathbf{\epsilon}}_{\theta}(\mathbf{x}_t, y, t) = \hat{\mathbf{\epsilon}}_{\theta}(\mathbf{x}_t, t) - \omega \sqrt{1 - \bar{\alpha}_t} \nabla_{\mathbf{x}_t} \log p_{\phi}(y | \mathbf{x}_t)$$

Controlling the generation process with classifier guidance

$$\begin{aligned}\tilde{\boldsymbol{\epsilon}}_{\theta}(\mathbf{x}_t, y, t) &= \hat{\boldsymbol{\epsilon}}_{\theta}(\mathbf{x}_t, t) - \omega \sqrt{1 - \bar{\alpha}_t} \nabla_{\mathbf{x}_t} \log p_{\phi}(y|\mathbf{x}_t) \\ &= -\sqrt{1 - \bar{\alpha}_t} (\nabla_{\mathbf{x}_t} \log p_{\phi}(\mathbf{x}_t) + \omega \nabla_{\mathbf{x}_t} \log p_{\phi}(y|\mathbf{x}_t)) \\ &= -\sqrt{1 - \bar{\alpha}_t} \nabla_{\mathbf{x}_t} \log p_{\phi}(\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log p_{\phi}(y|\mathbf{x}_t) \omega \\ &= -\sqrt{1 - \bar{\alpha}_t} \nabla_{\mathbf{x}_t} \log(p_{\phi}(\mathbf{x}_t) p_{\phi}(y|\mathbf{x}_t) \omega)\end{aligned}$$

Controlling the generation process with classifier guidance

- (+) Not only is the generation quality improved, but **conditional generation** with labels also becomes possible!
- (−) Additional training of a classifier is required.

Controlling the generation process with classifier guidance



$$\omega = 1$$



$$\omega = 10$$

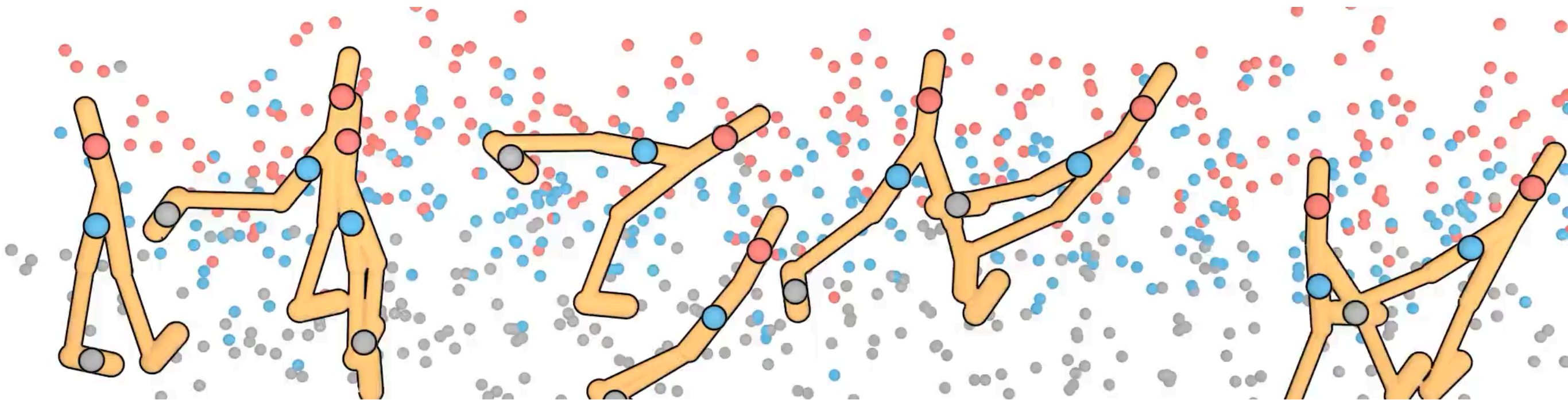
Planning with Diffusion for Flexible Behavior Synthesis

Michael Janner^{*}, Yilun Du^{*}, Joshua Tenenbaum, and Sergey Levine

Model-based RL with diffusion models:

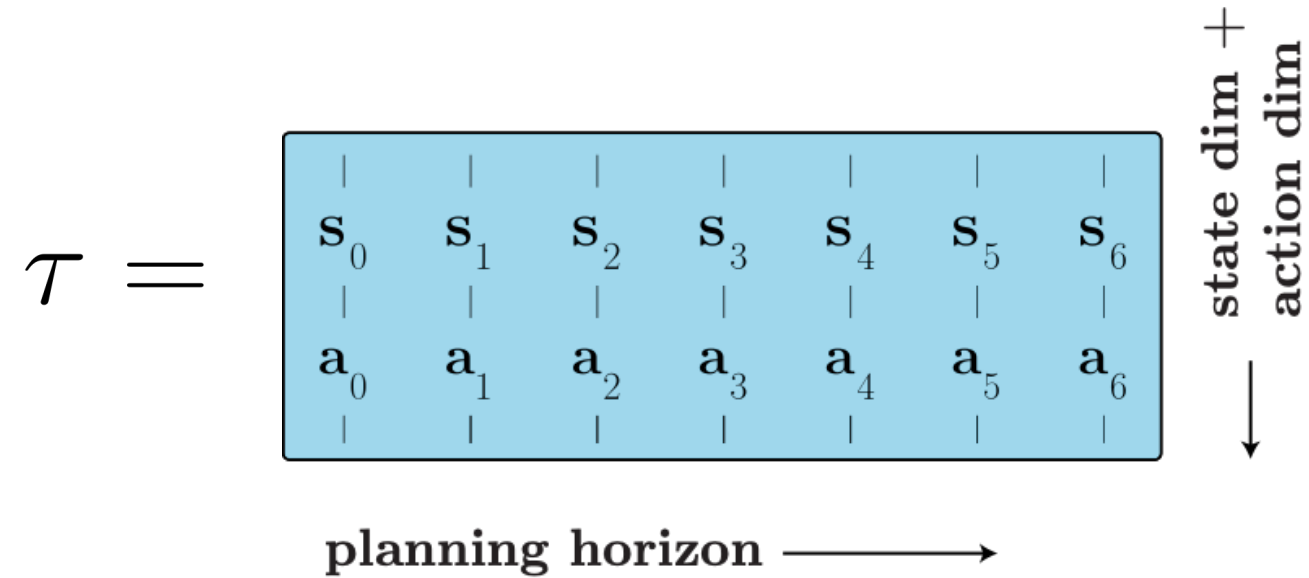
1. Train a state-action **trajectory** diffusion model.
2. Guide it with learnt value functions towards high value trajectory samples

1. Train a state-action **trajectory** diffusion model.



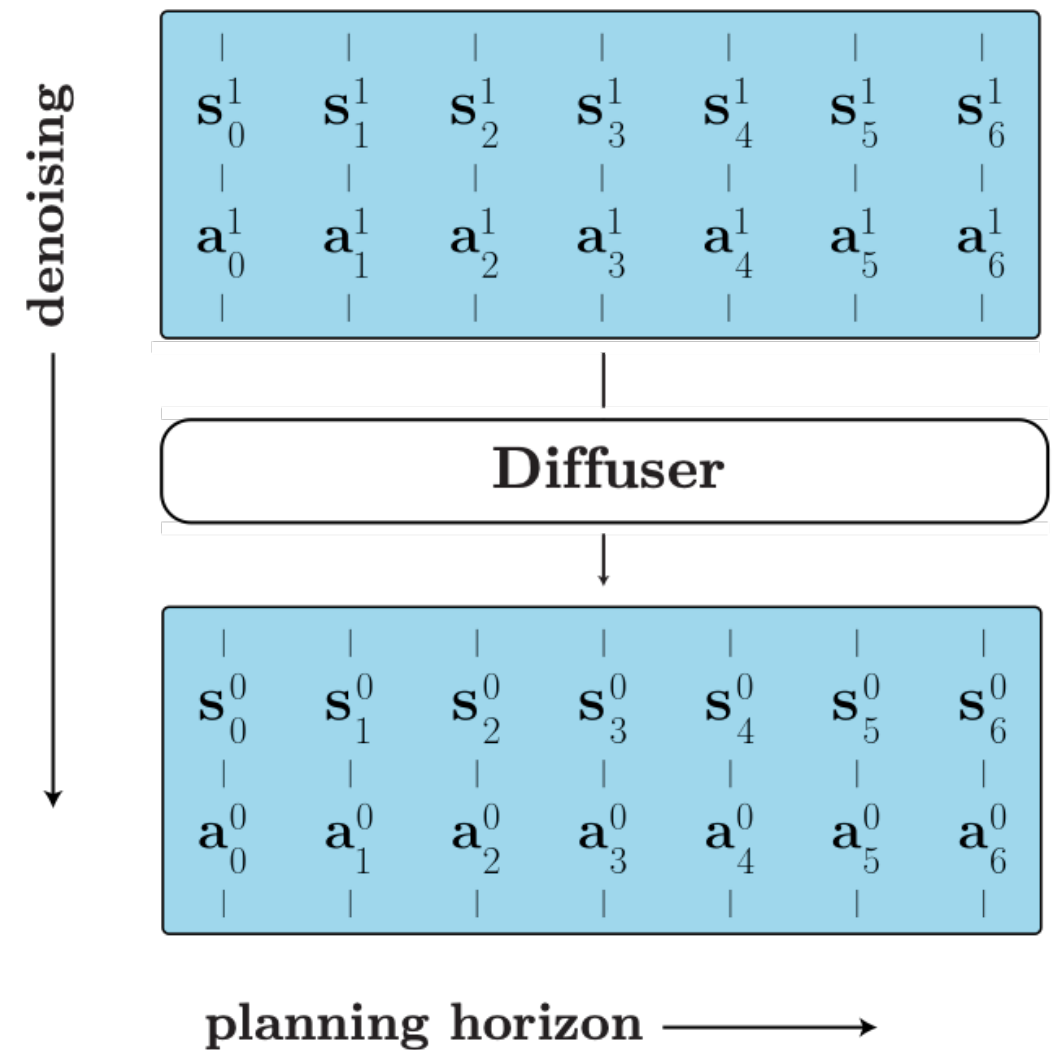
A generative model of trajectories

- Represent trajectories as single-channel images



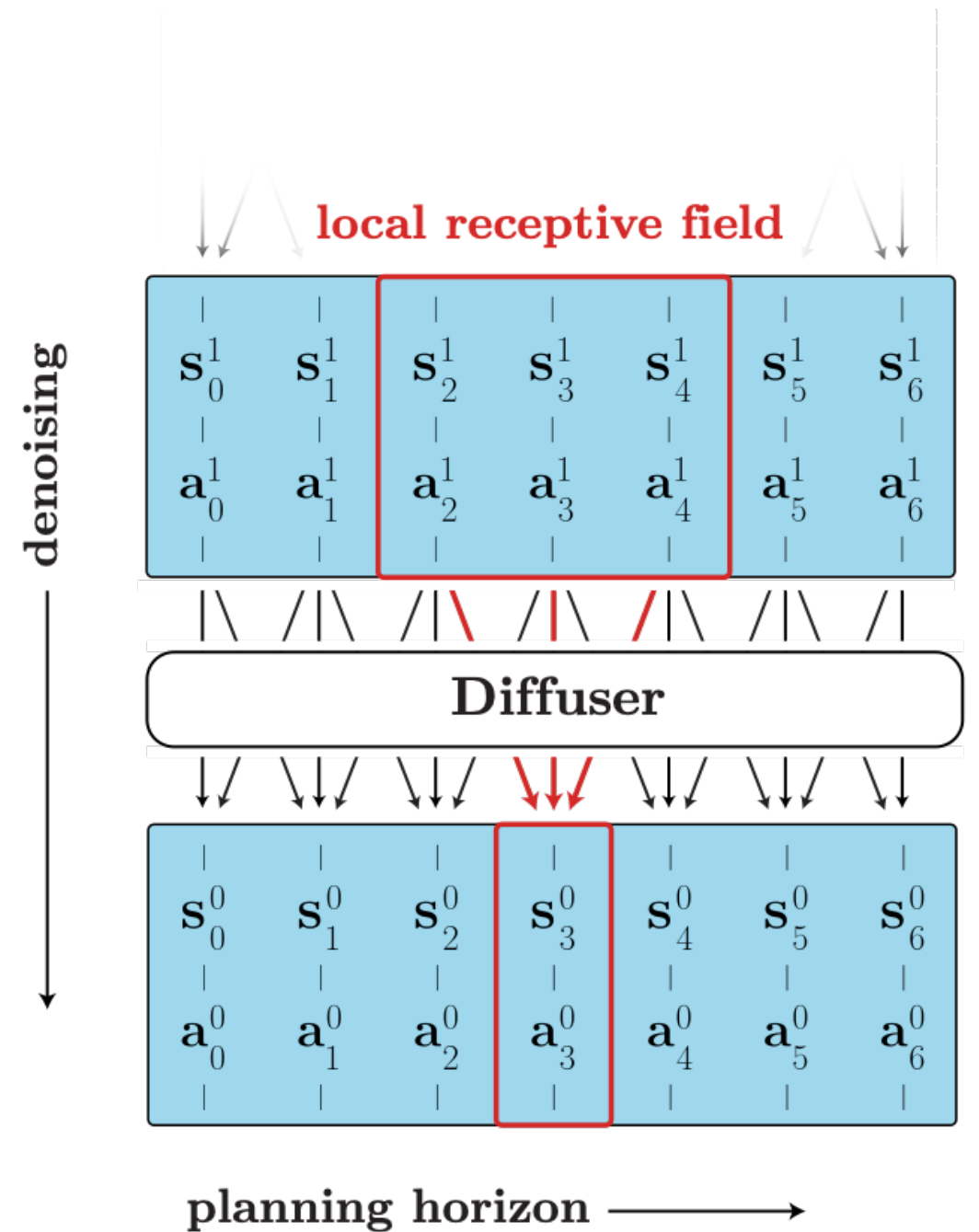
A generative model of trajectories

- Represent trajectories as single-channel images
- Train a diffusion model to iteratively denoise entire trajectory



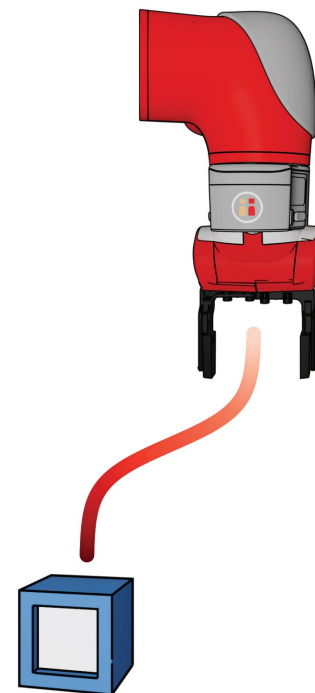
A generative model of trajectories

- Represent trajectories as single-channel images
- Train a diffusion model to iteratively denoise entire trajectory
- Use (one-dimensional) convolutions for temporal equivariance and horizon-independence



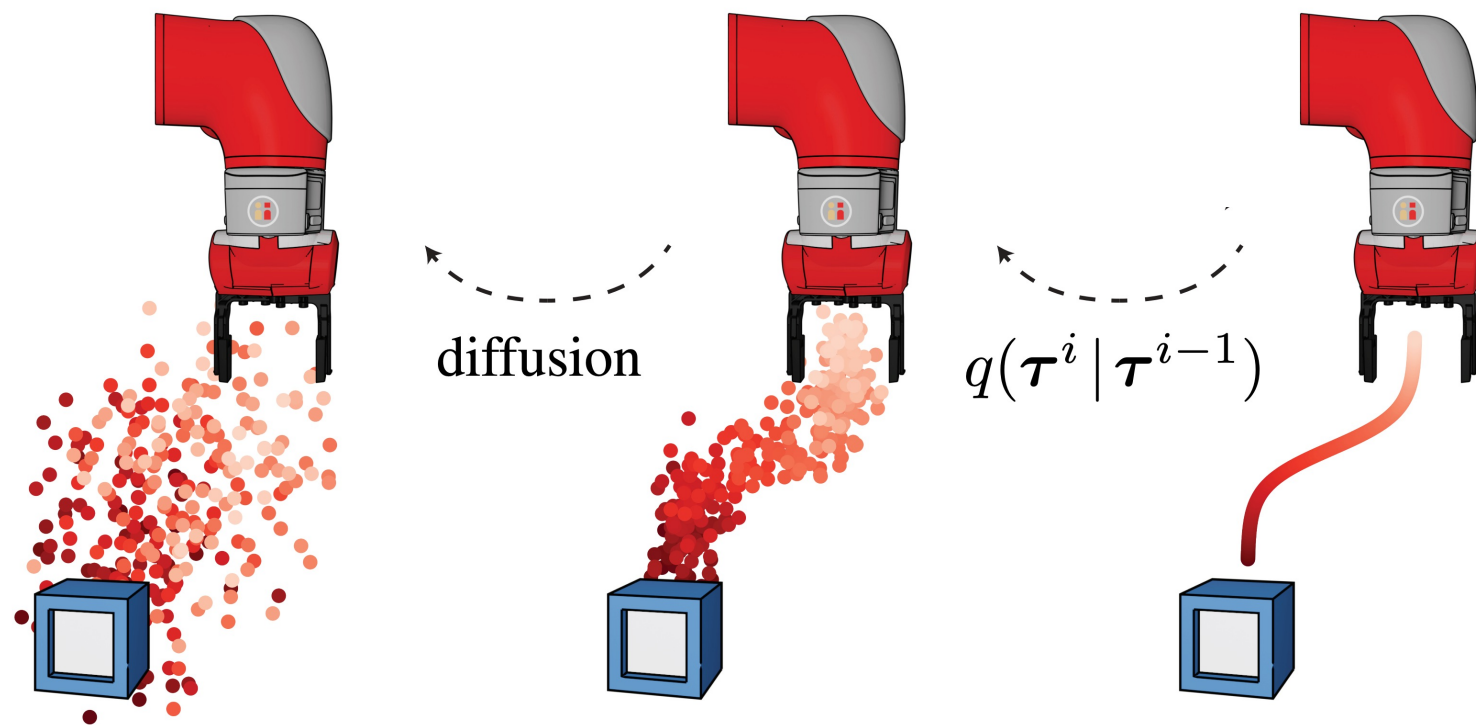
Non-autoregressive prediction

- Prediction is **non-autoregressive**: entire trajectory is predicted simultaneously



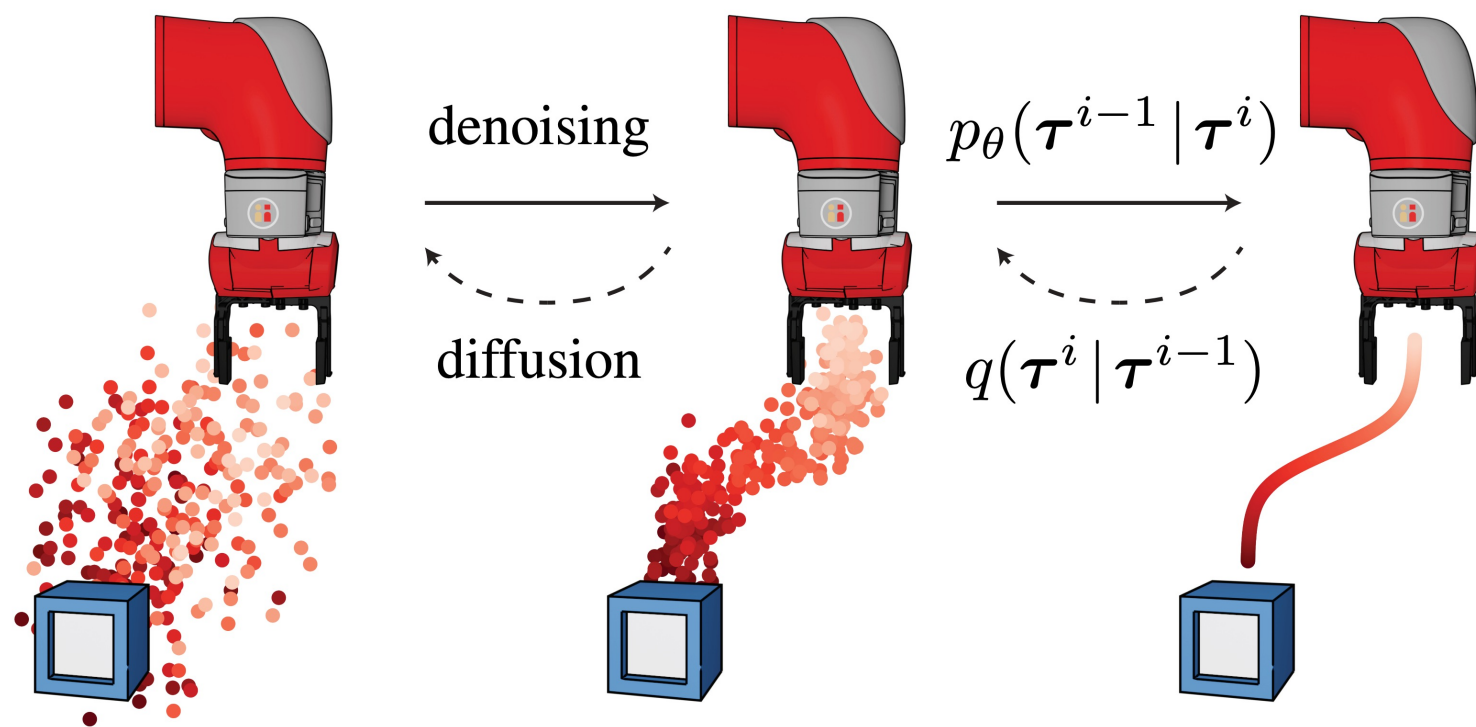
Non-autoregressive prediction

- Prediction is **non-autoregressive**: entire trajectory is predicted simultaneously



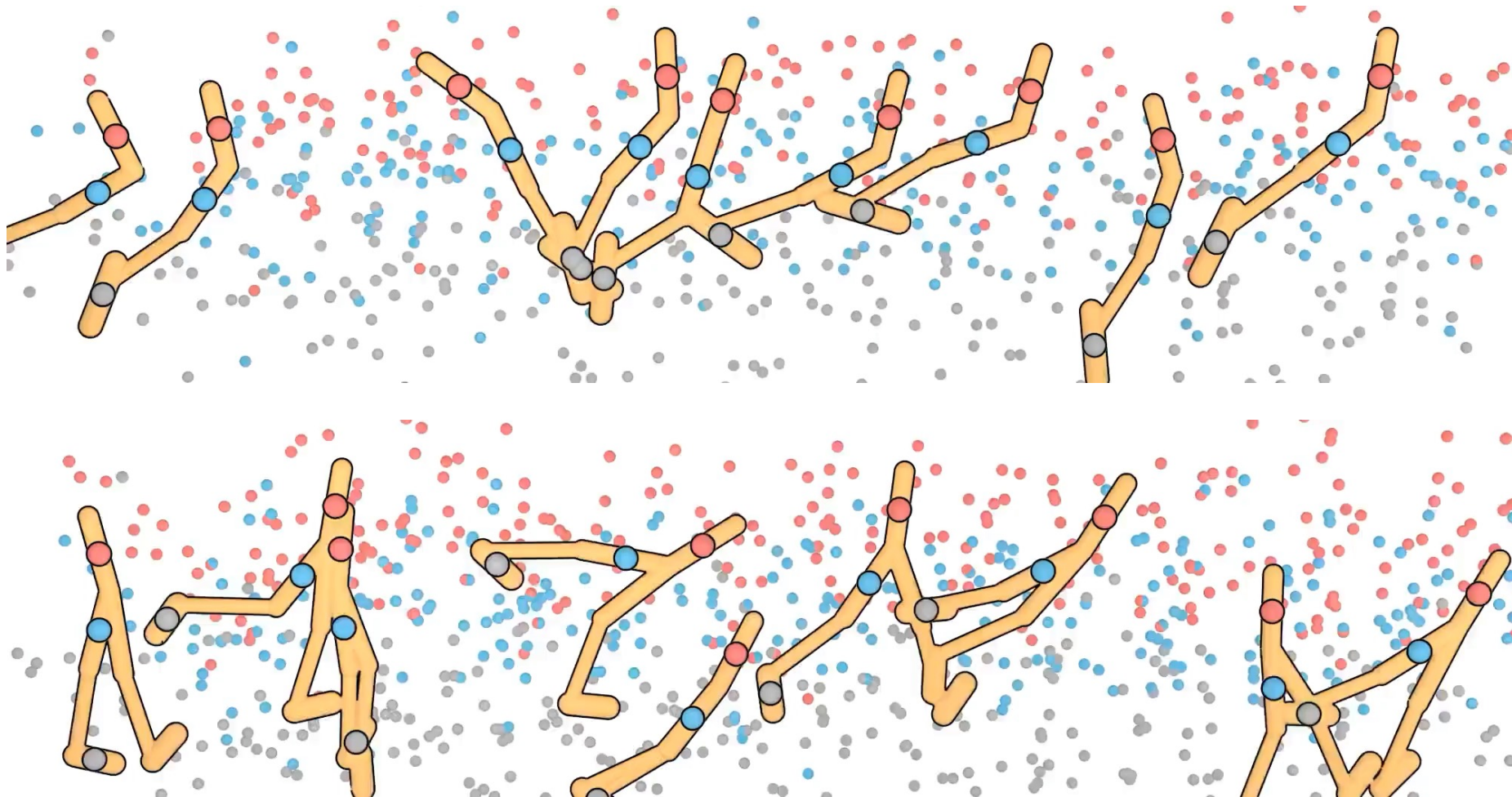
Non-autoregressive prediction

- Prediction is **non-autoregressive**: entire trajectory is predicted simultaneously

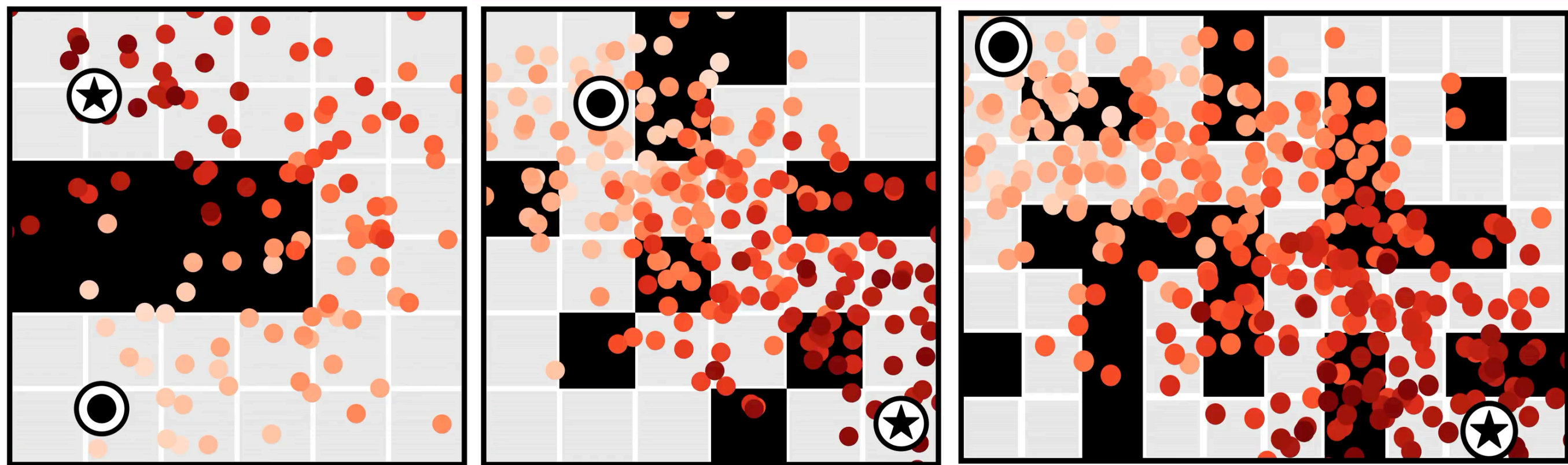


Sampling from Diffuser

- Sampling occurs by iteratively refining randomly-initialized trajectories

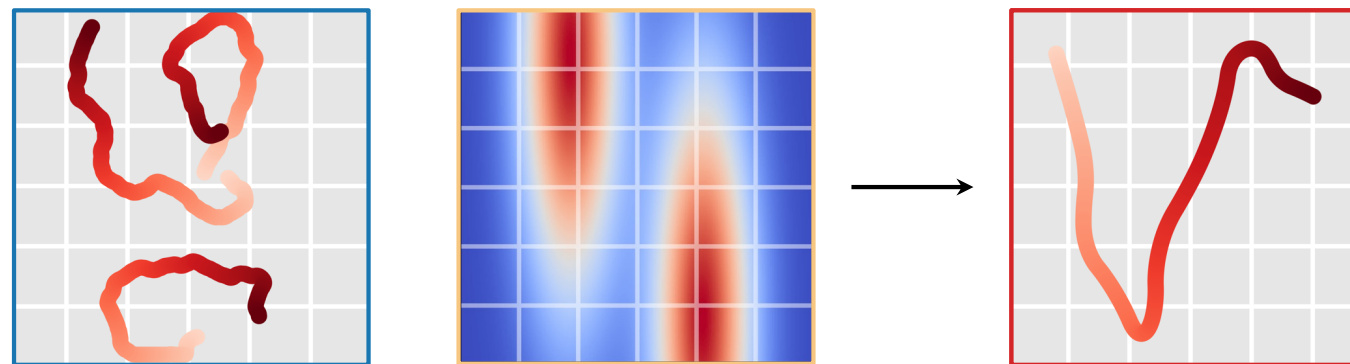


2. Guide the trajectory diffusion models with learnt value/cost functions



2. Guide the trajectory diffusion models with learnt value/cost functions

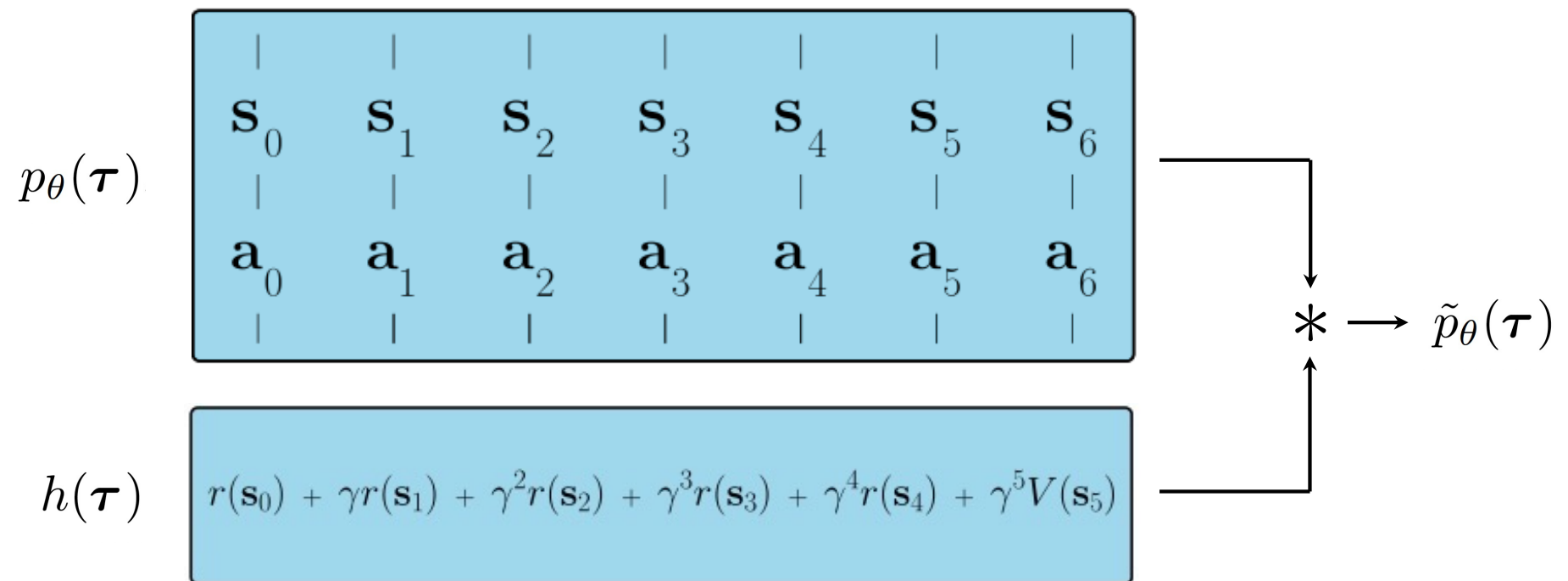
$$\underbrace{\tilde{p}_{\theta}(\tau)}_{\text{Behavior Model}} \propto \underbrace{p_{\theta}(\tau)}_{\text{Diffusion Model}} \underbrace{h(\tau)}_{\text{Guidance Function}}$$



See also: Model Based Planning with Energy Based Models: Du et al, CoRL 2019.

Offline Reinforcement Learning through Value Guidance

- Can use a value function to bias trajectory model to particular task



- Can use a single Diffuser model for multiple different tasks

Goal Planning through Inpainting

- Specify a guidance function over the first and goal state of a trajectory

Goal Planning through Inpainting

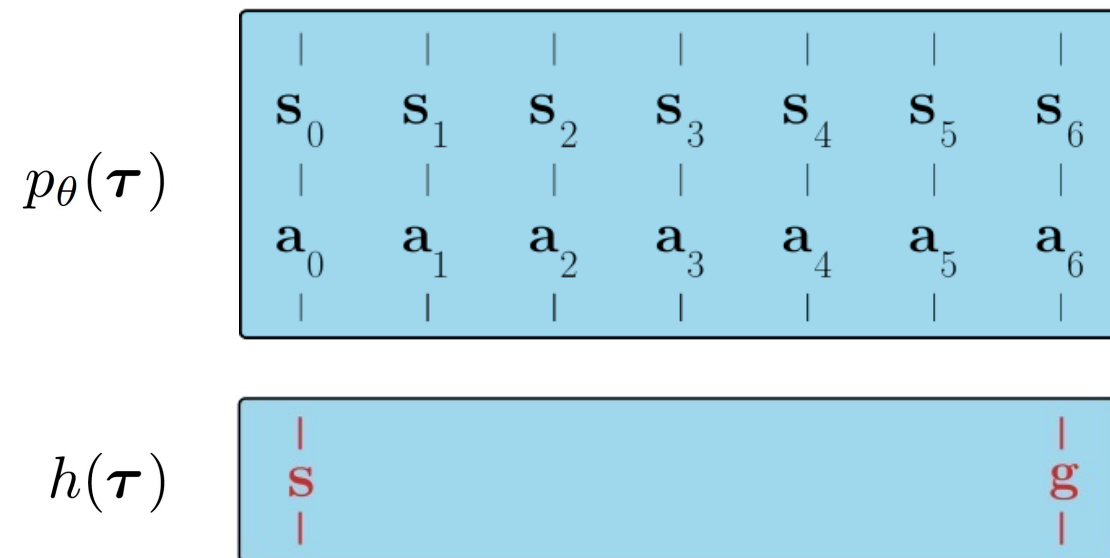
- Specify a guidance function over the first and goal state of a trajectory

$p_{\theta}(\boldsymbol{\tau})$

$\mathbf{s}_0$	$\mathbf{s}_1$	$\mathbf{s}_2$	$\mathbf{s}_3$	$\mathbf{s}_4$	$\mathbf{s}_5$	$\mathbf{s}_6$
$\mathbf{a}_0$	$\mathbf{a}_1$	$\mathbf{a}_2$	$\mathbf{a}_3$	$\mathbf{a}_4$	$\mathbf{a}_5$	$\mathbf{a}_6$

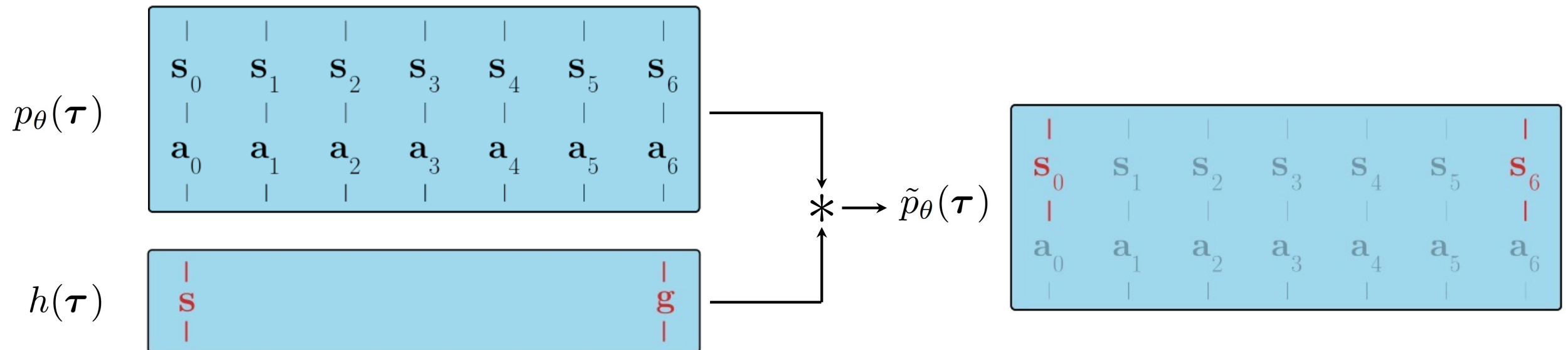
Goal Planning through Inpainting

- Specify a guidance function over the first and goal state of a trajectory



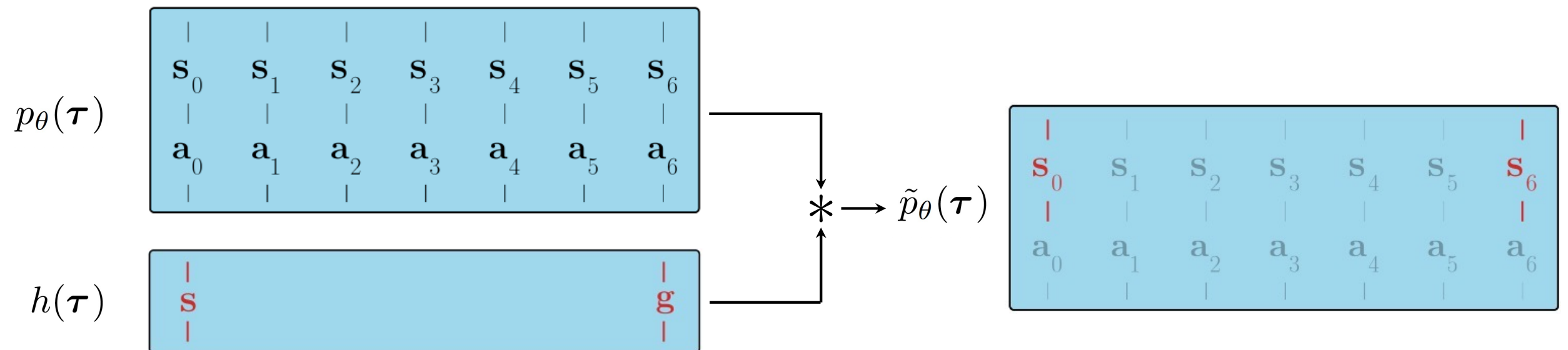
Goal Planning through Inpainting

- Specify a guidance function over the final explicit goal state of a trajectory



Goal Planning through Inpainting

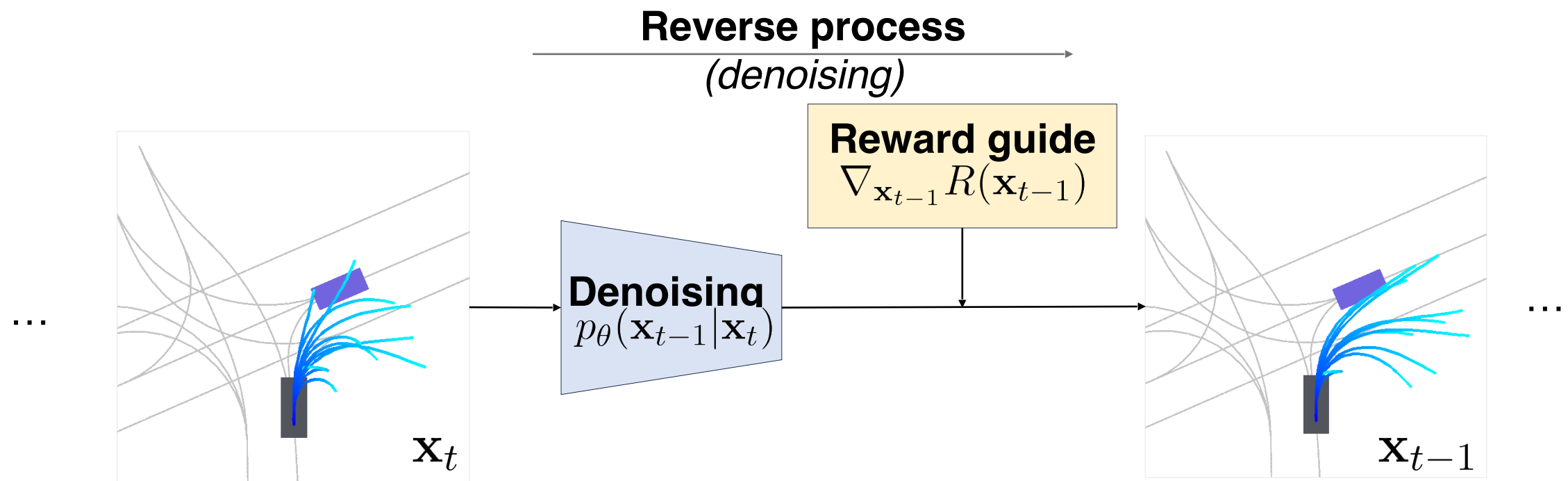
- Specify a guidance function over the final explicit goal state of a trajectory



- Construct a goal seeking policy through guidance

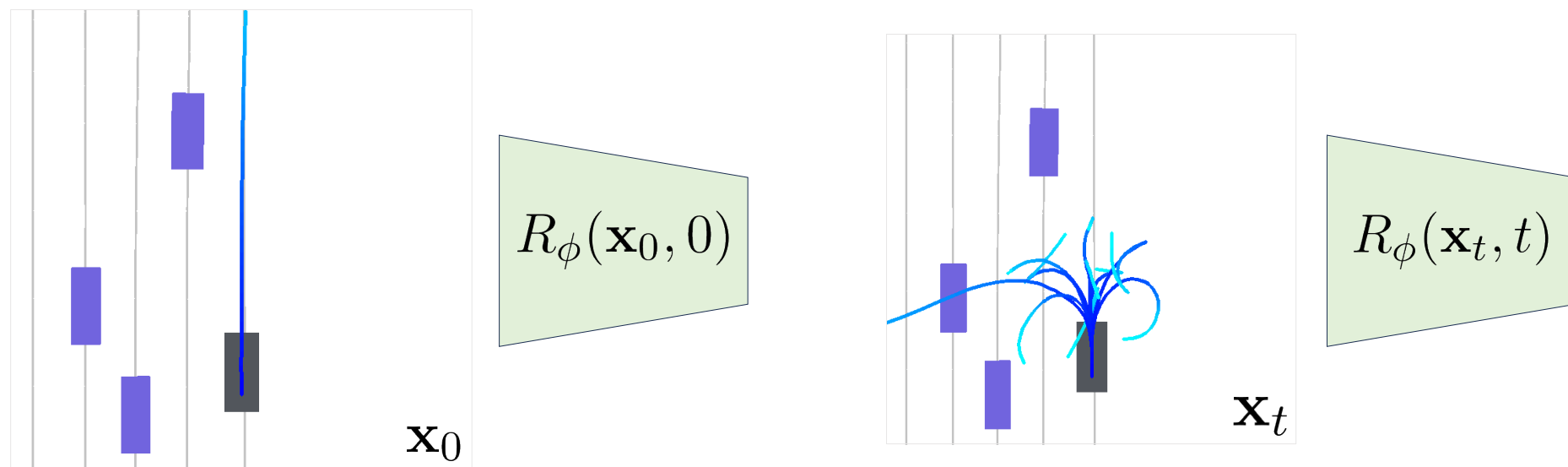
Planning with diffusion models: Differentiable reward guidance

We alternate between **denoising steps** and **reward-gradient guidance steps**

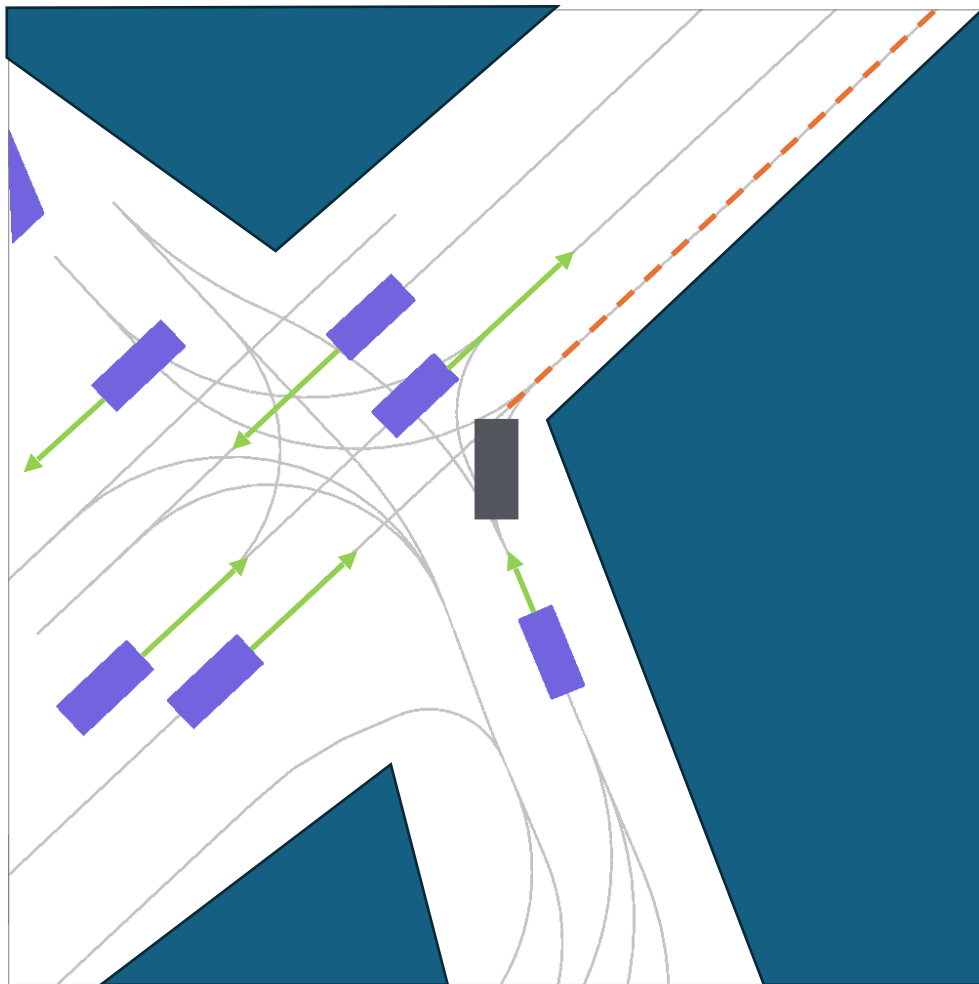


Challenges with reward-gradient guidance

- Reward functions should be differentiable
- Reward functions should be fitted in clean and noised data
- Fitted reward functions can be exploited



The driving reward function



Collision avoidance

Drivable area compliance

Route progress

Time-to-collision

Comfort

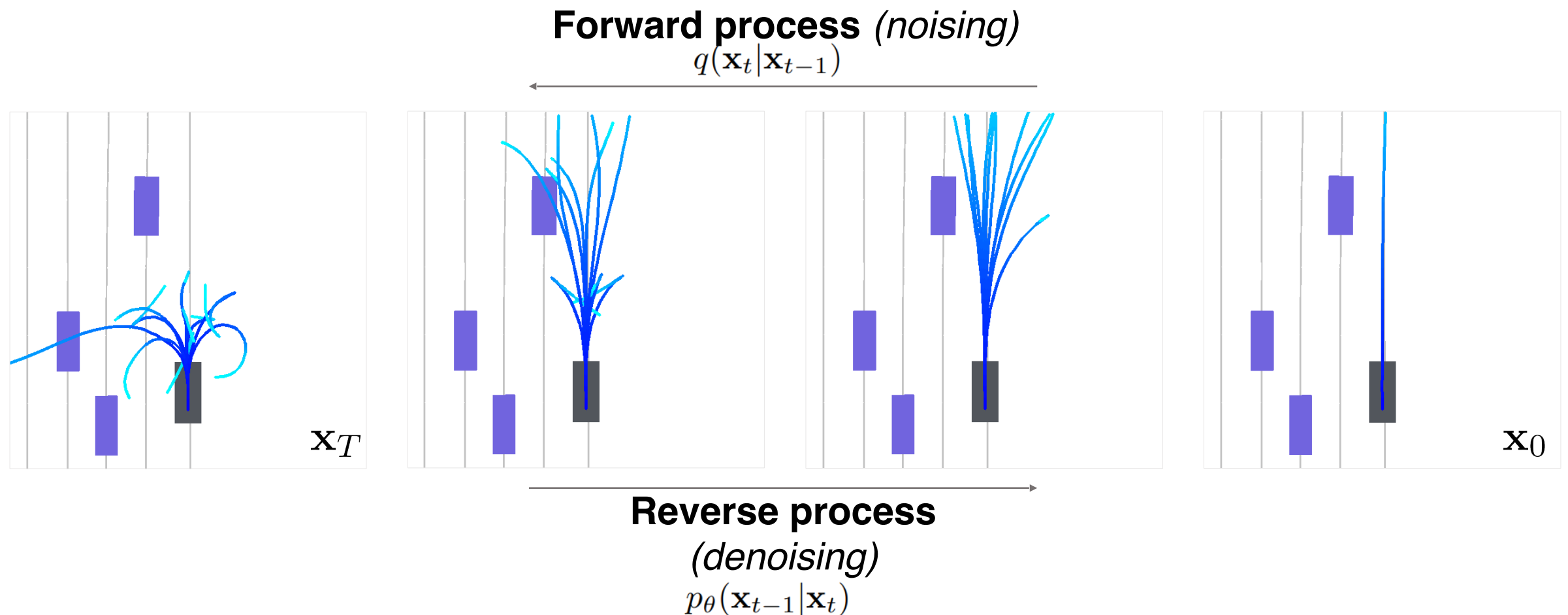
Speed limit compliance

Trackability ← *Non-differentiable!*

Generative Evolutionary Search (Diffusion-ES)

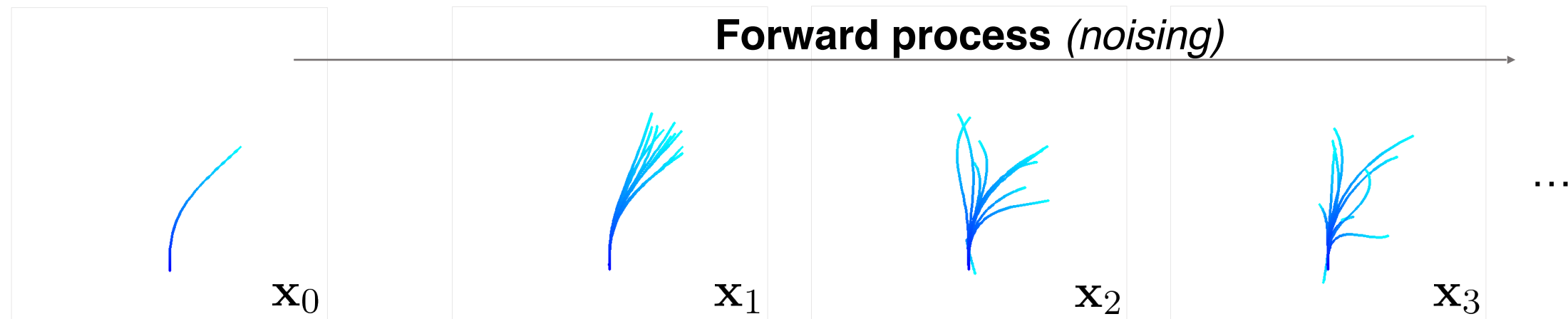
Diffusion + Evolutionary Search

Modeling trajectories with diffusion models

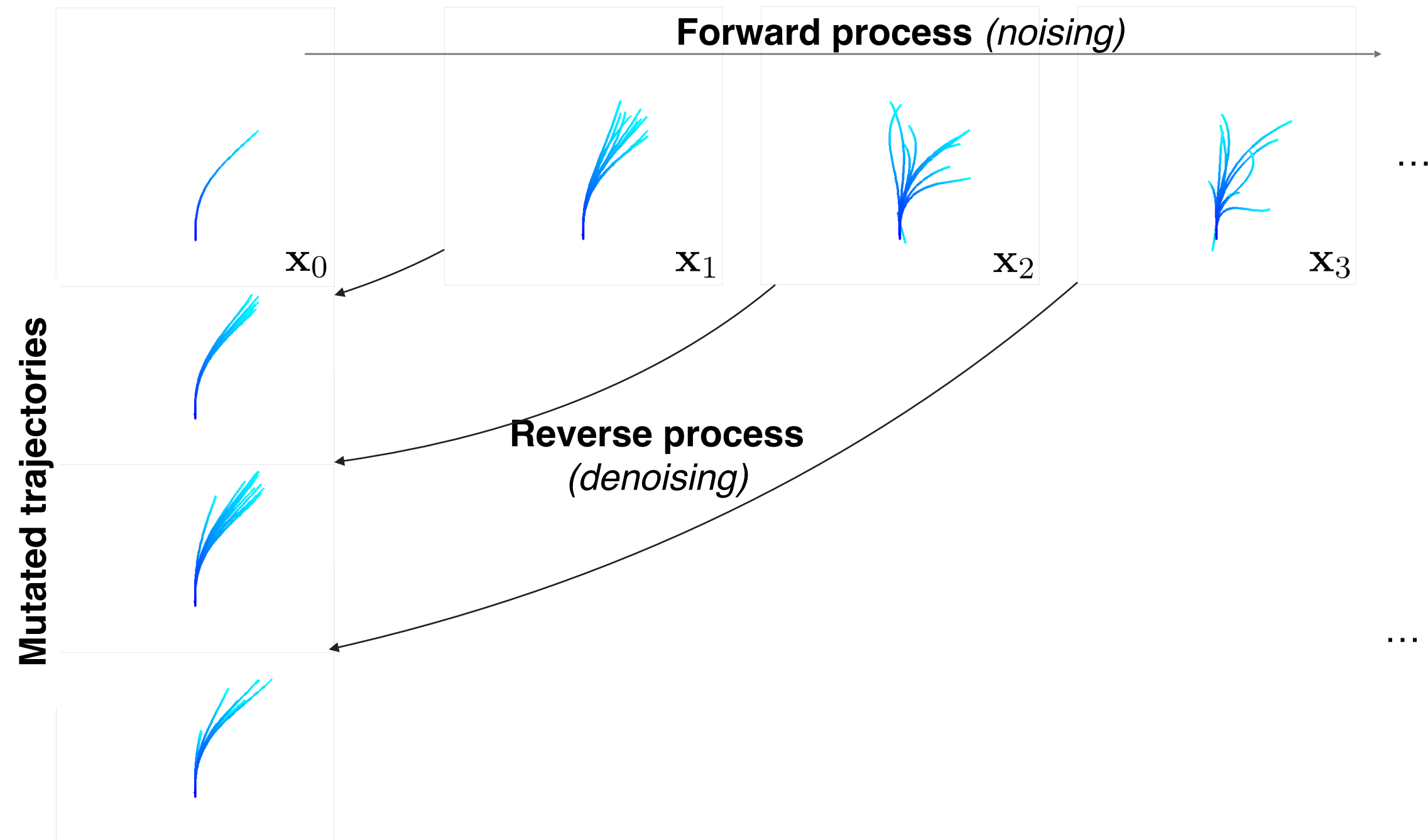


How do we use this for decision making?

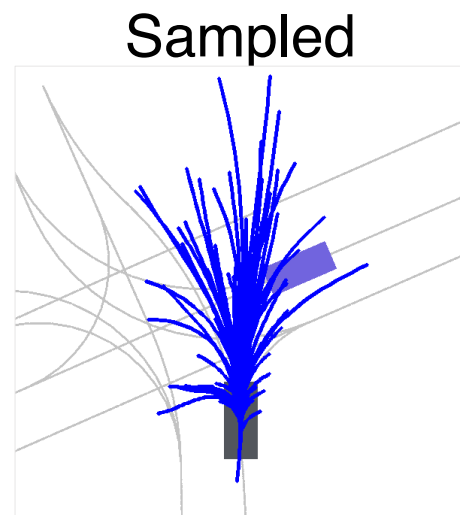
Mutation via truncated diffusion



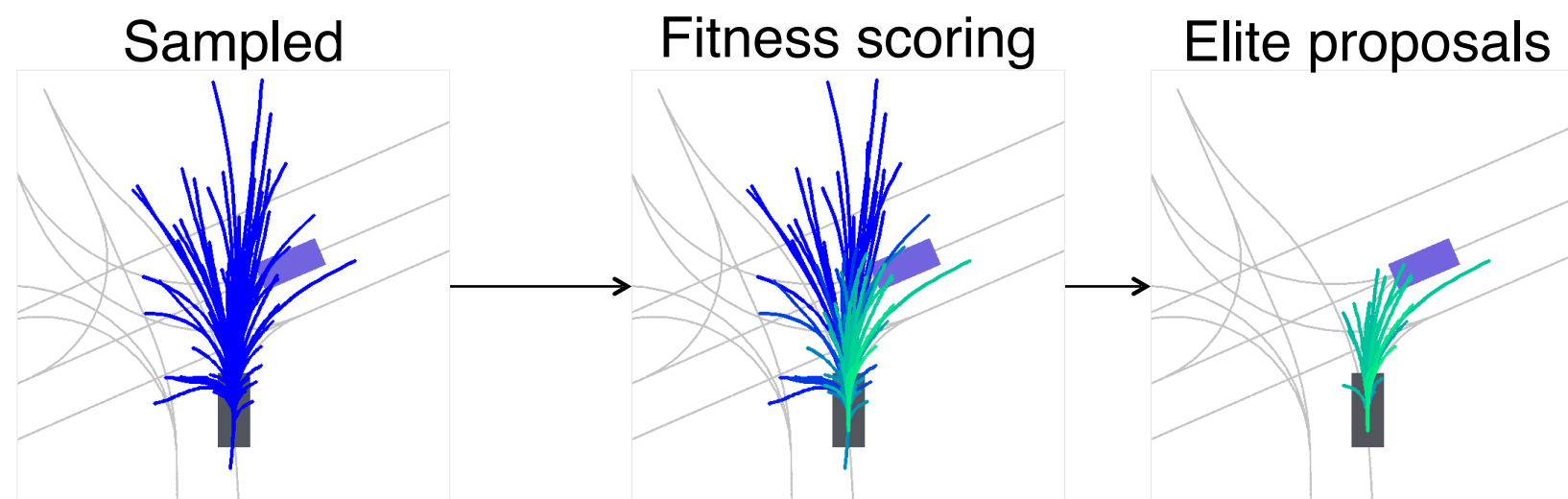
Mutation via truncated diffusion



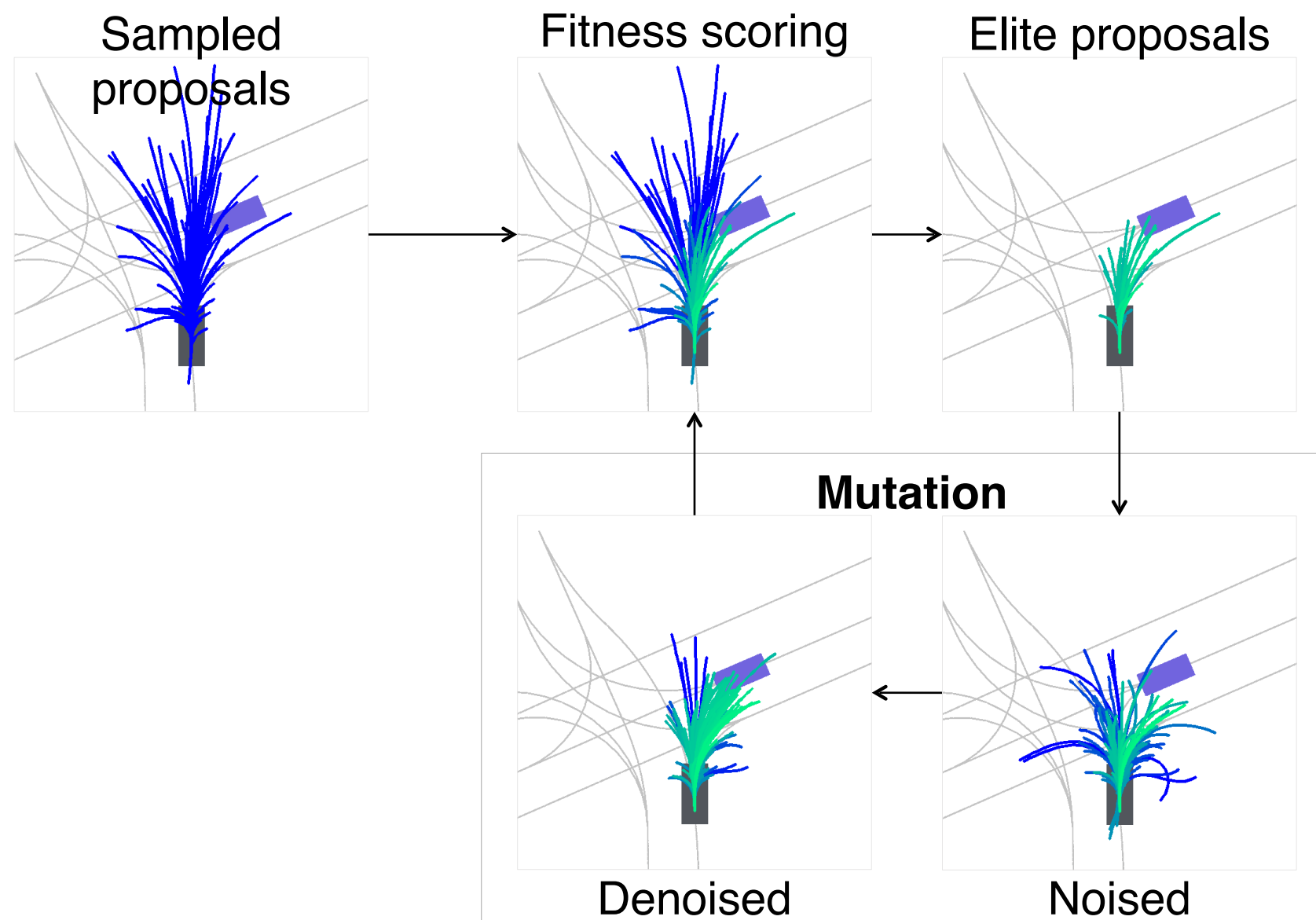
Diffusion-ES



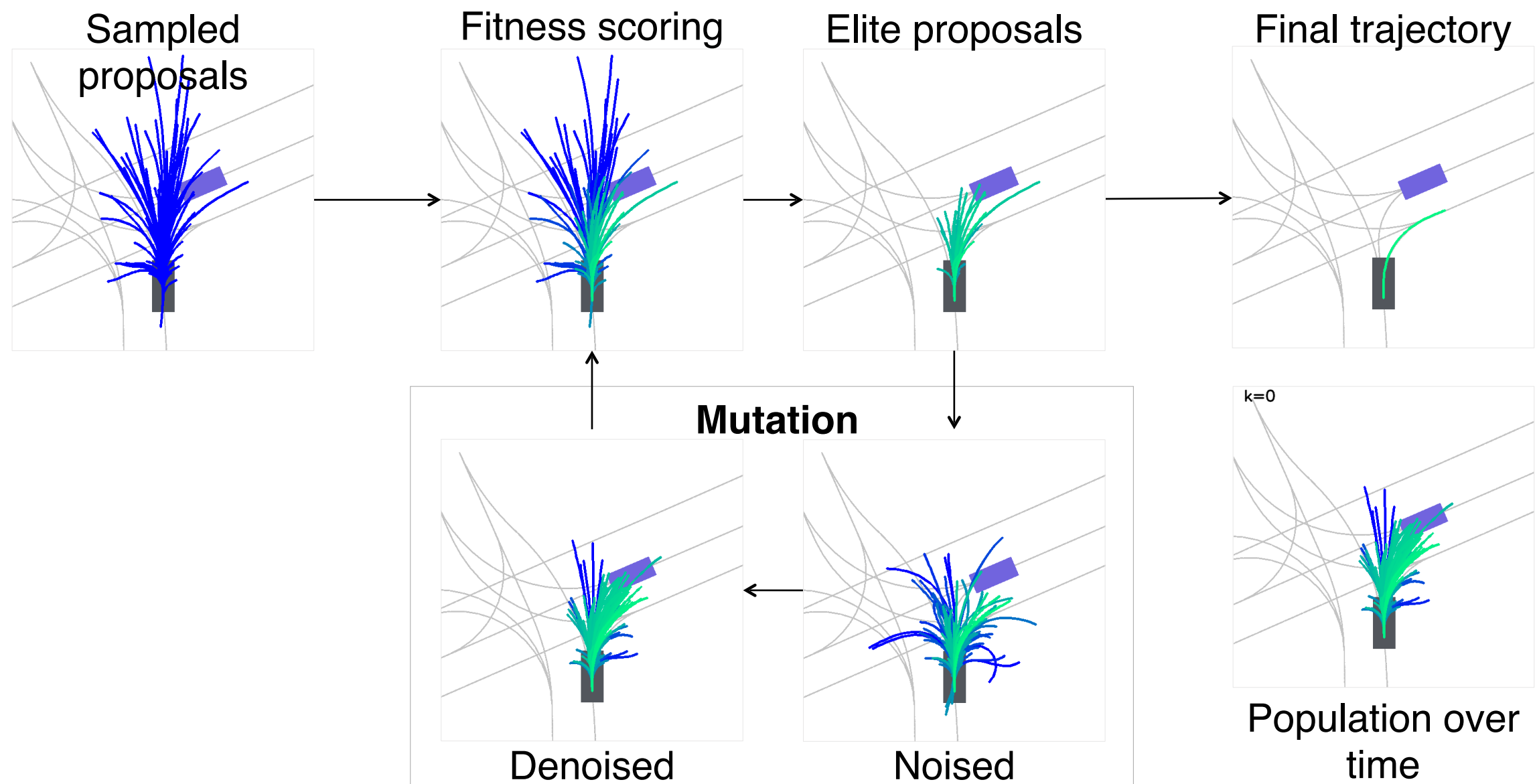
Diffusion-ES



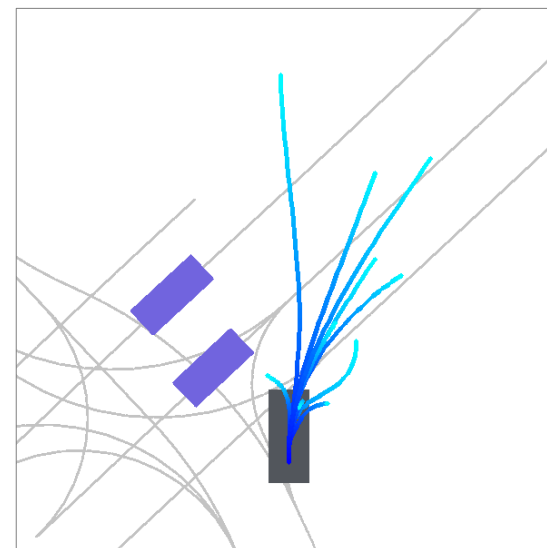
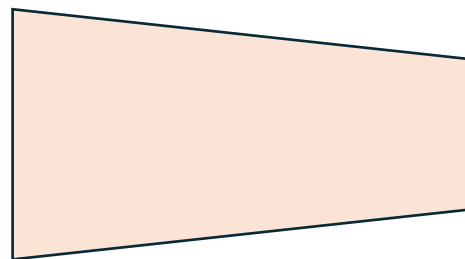
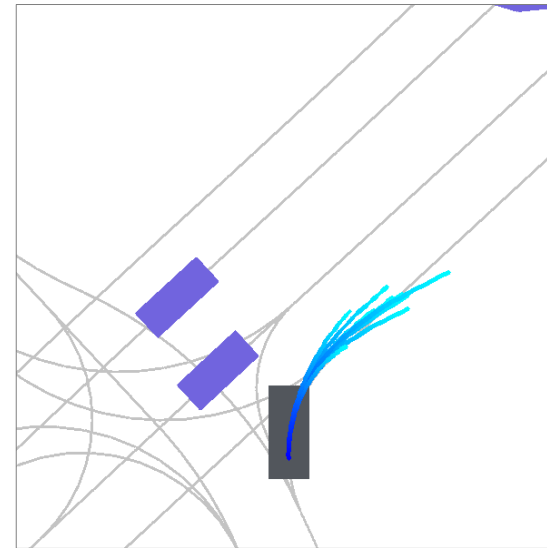
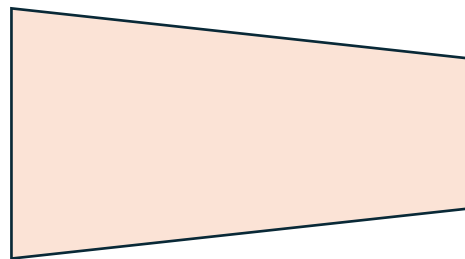
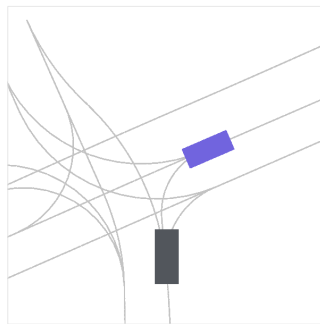
Diffusion-ES



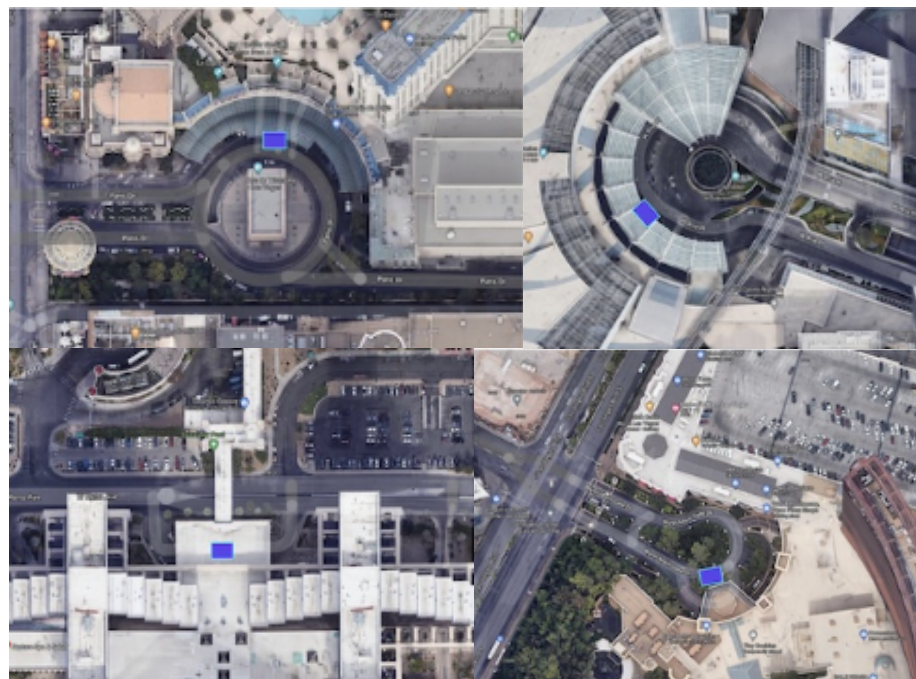
Diffusion-ES



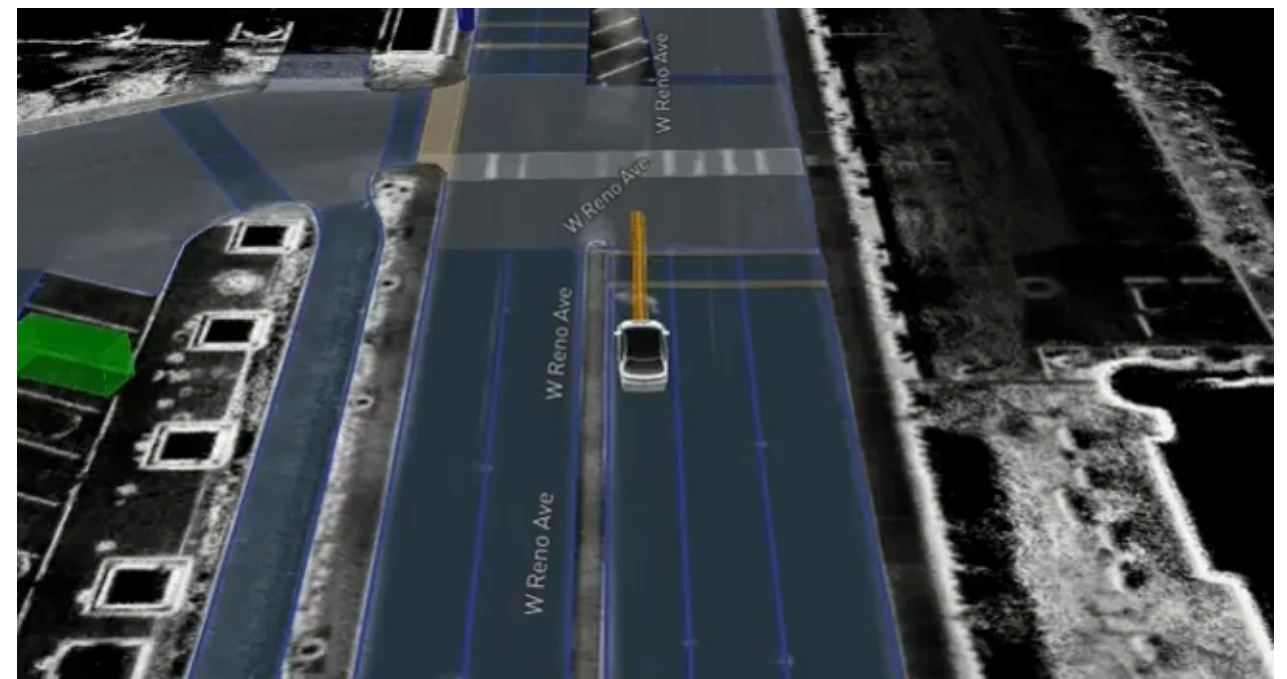
The more conditioning, the narrower the distribution



nuPlan



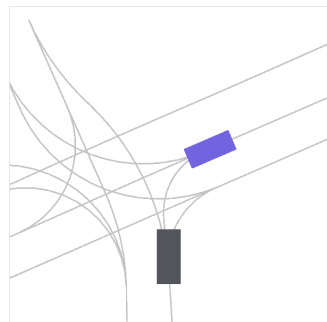
~1200 hours of **real-world** driving logs
Dataset labeled using offline perception
Data-driven **closed-loop** simulation



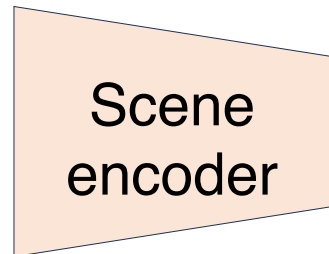
Other agents are **reactive**
Includes vehicles, pedestrians, cyclists, traffic
lights, and construction zones

Scene-conditioned diffusion policies

Scene context



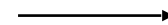
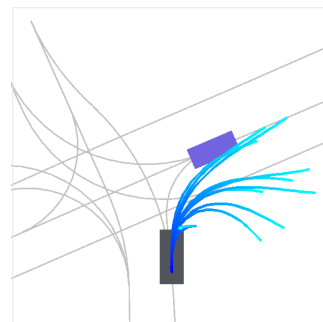
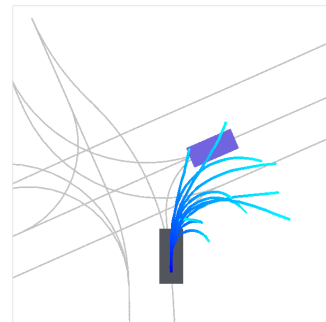
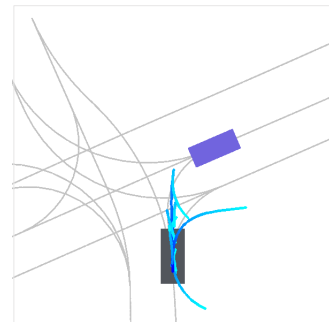
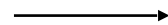
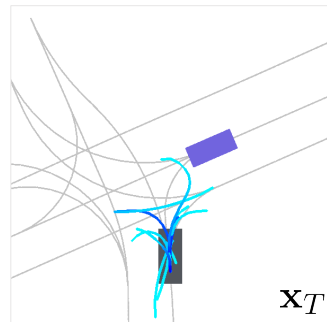
Scene encoder



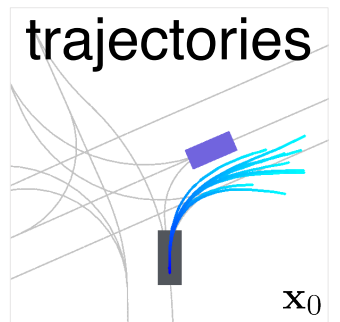
Reverse process (*denoising*)

$$p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t, c)$$

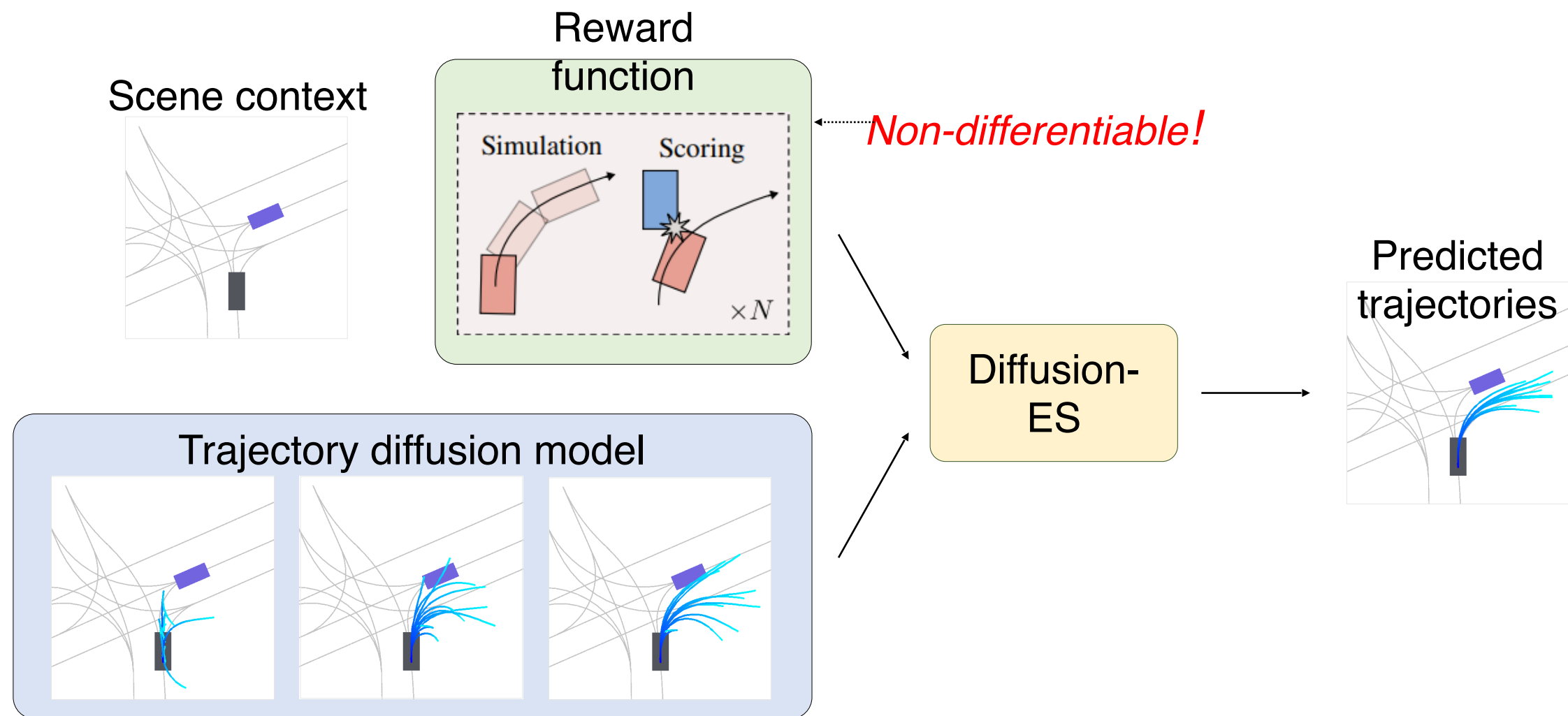
Random noise



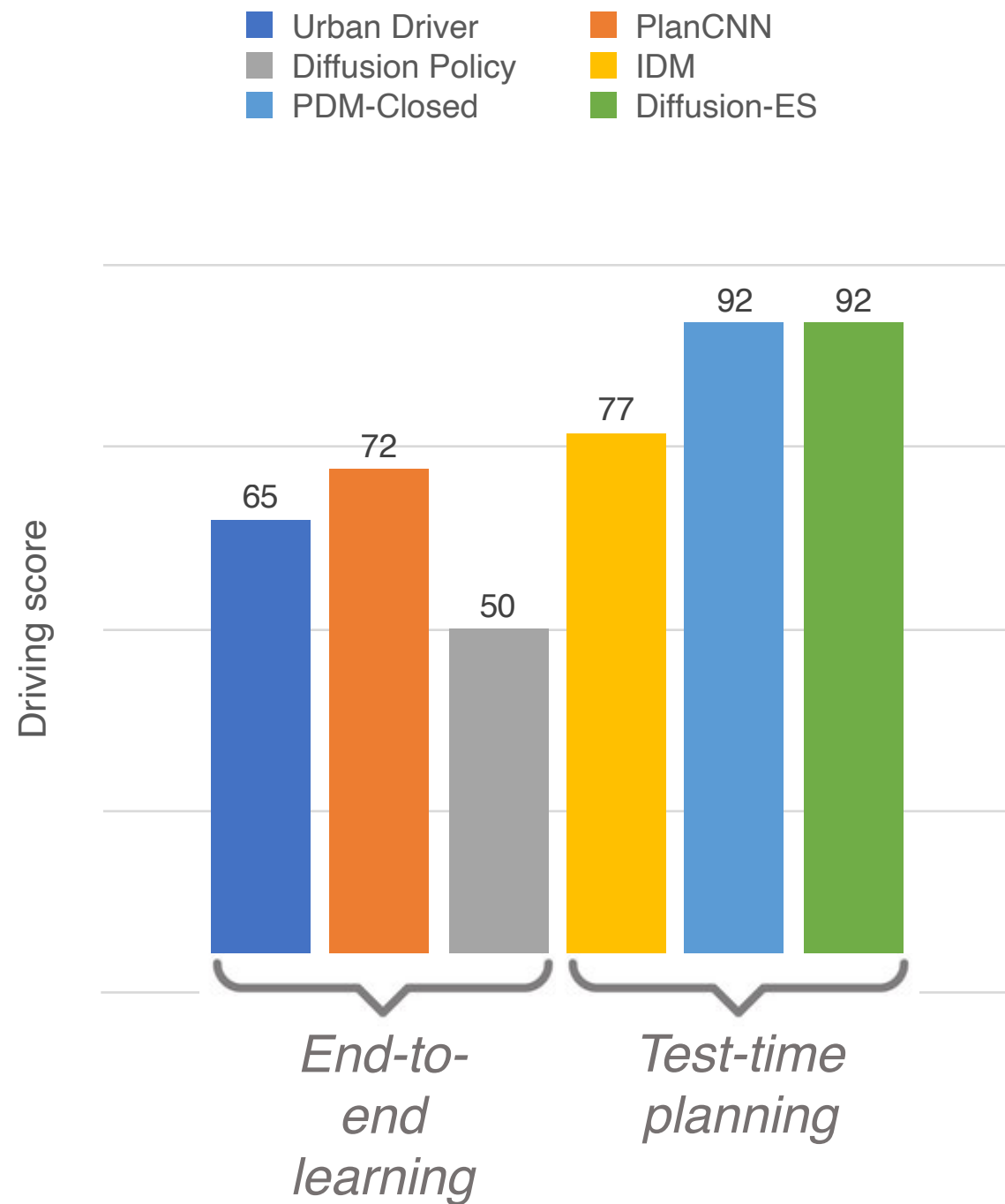
Predicted trajectories



Our approach in nuPlan



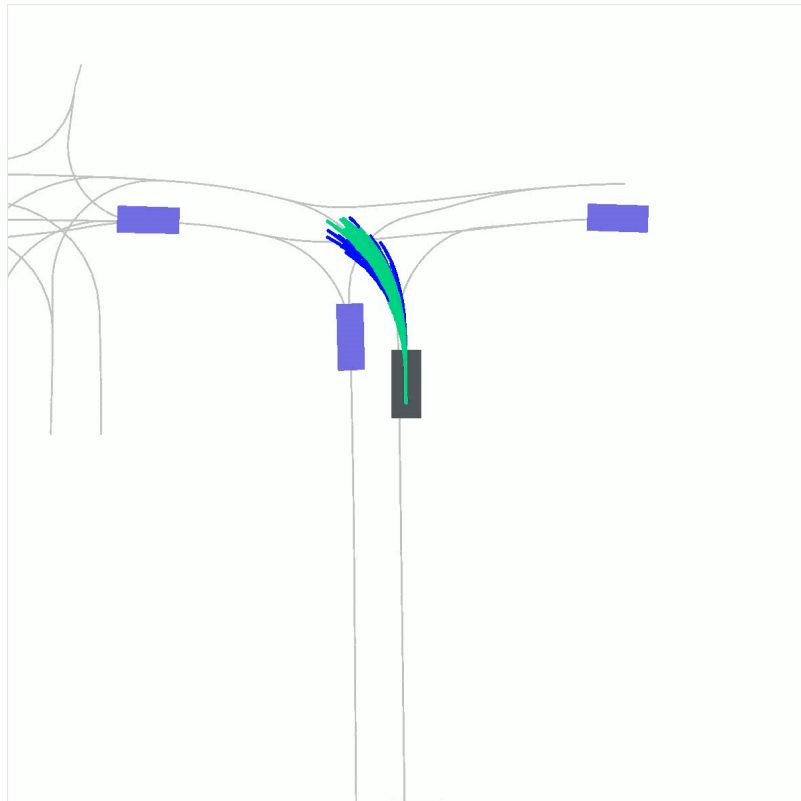
nuPlan benchmark performance



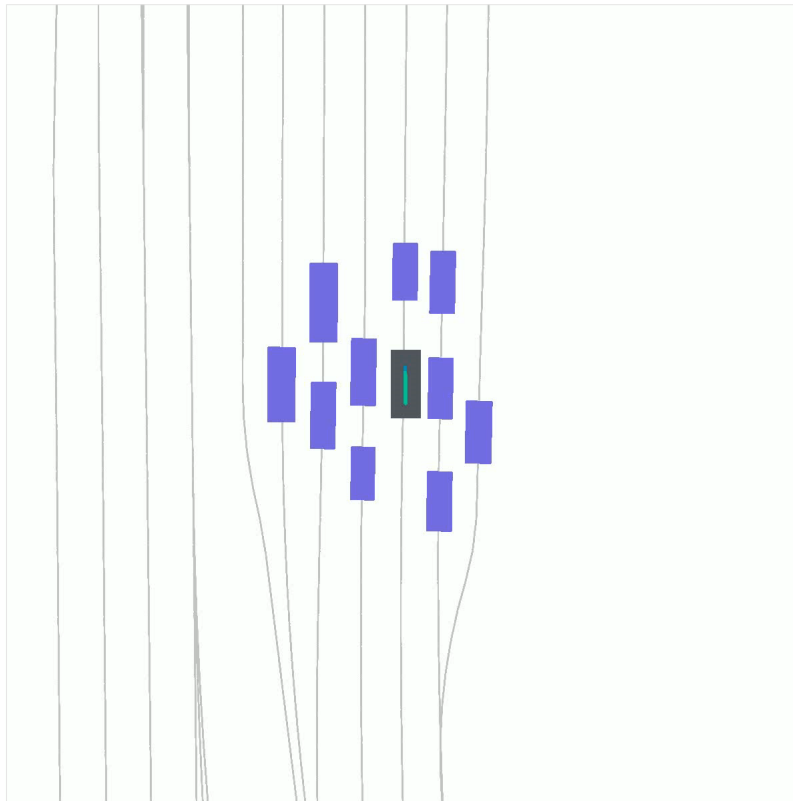
Test-time planning consistently outperforms reactive policies on nuPlan

Diffusion-ES matches the state-of-the-art without any explicit scene conditioning

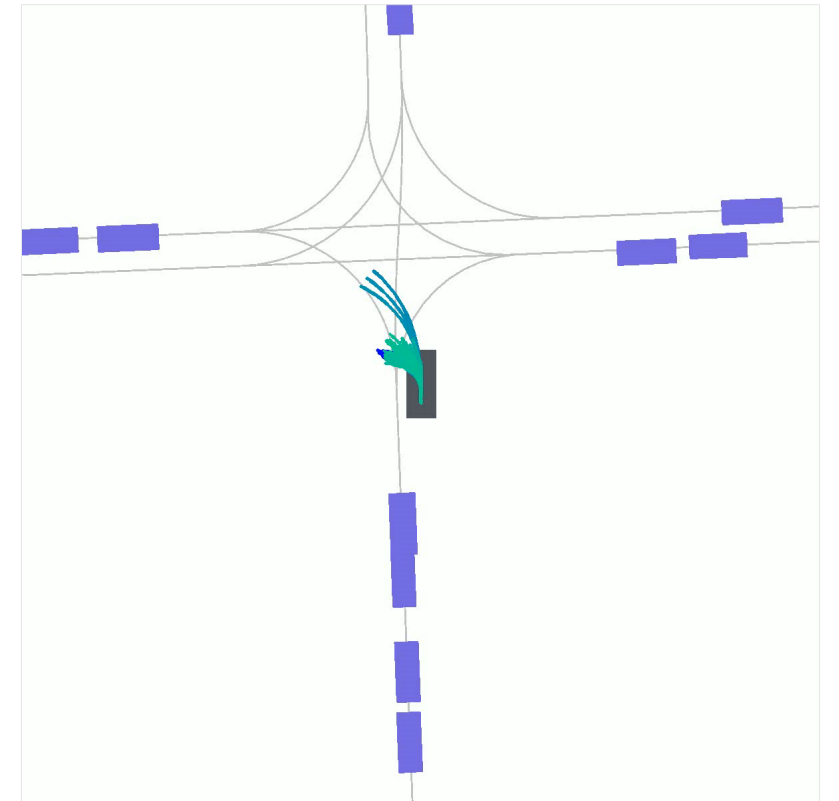
Learned driving behaviors



Yielding to incoming cars



Navigating dense traffic



Making unprotected turns